

图像滤波器及其应用

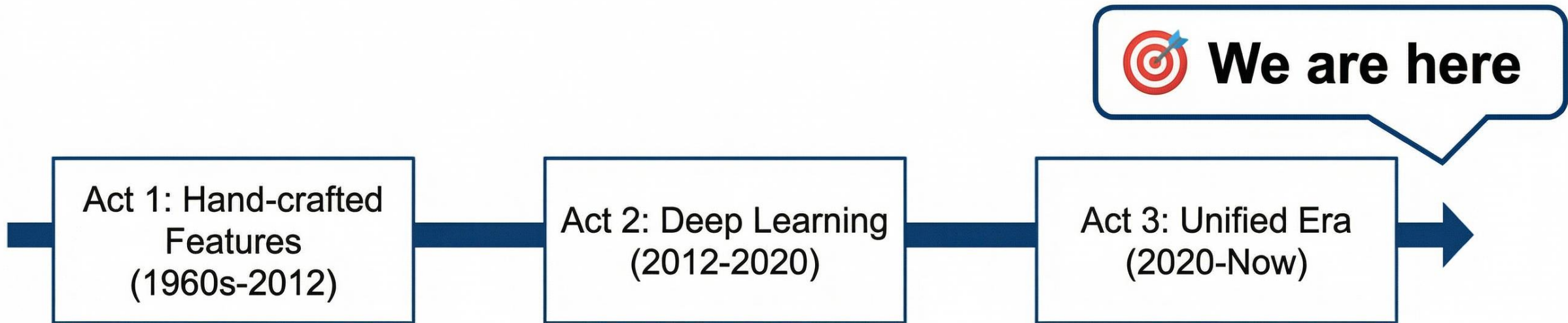
讲师: Hao WANG
AI Thrust, 信息枢纽



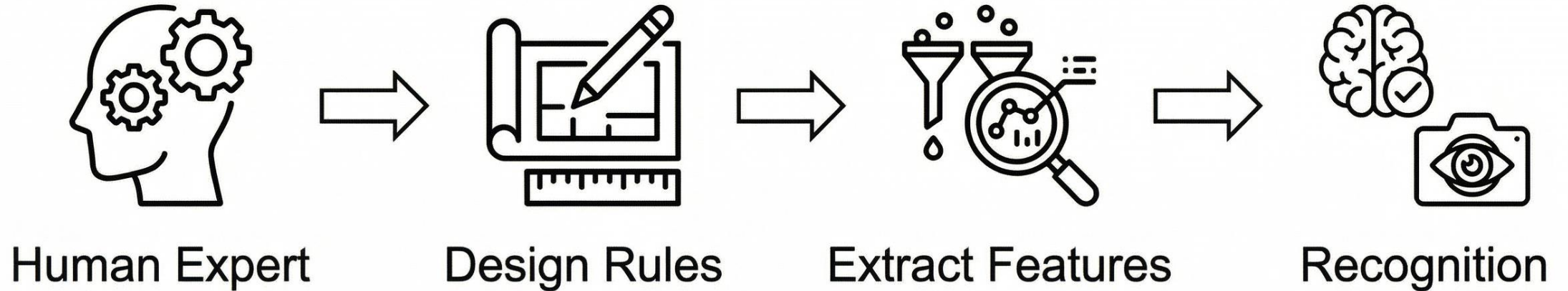
上周回顾



Three Eras of CV



Hand-crafted Approach



Philosophy: Humans tell machines

Examples: SIFT, HOG, Haar

Limitations

✗ Task-specific

✗ Poor generalization

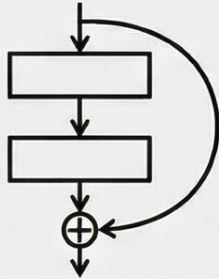
✗ Hard to design

✗ Performance ceiling

Breakthrough needed...

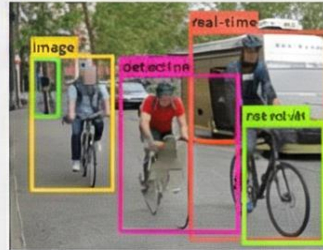


DL Achievements (2012-2020)



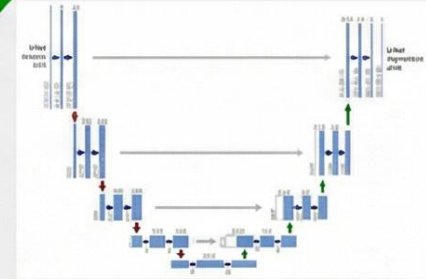
ResNet:

Classification: 95%+



YOLO:

Detection: Real-time



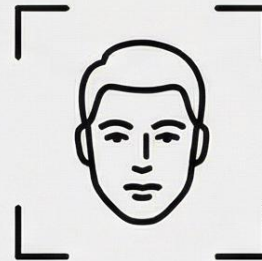
U-Net:

Segmentation: Pixel-level



OpenPose:

Pose: Joints



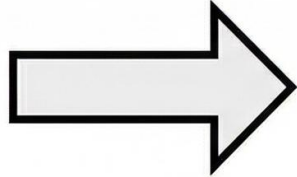
FaceNet:

Face: Beyond human

+ Depth, Video, 3D...

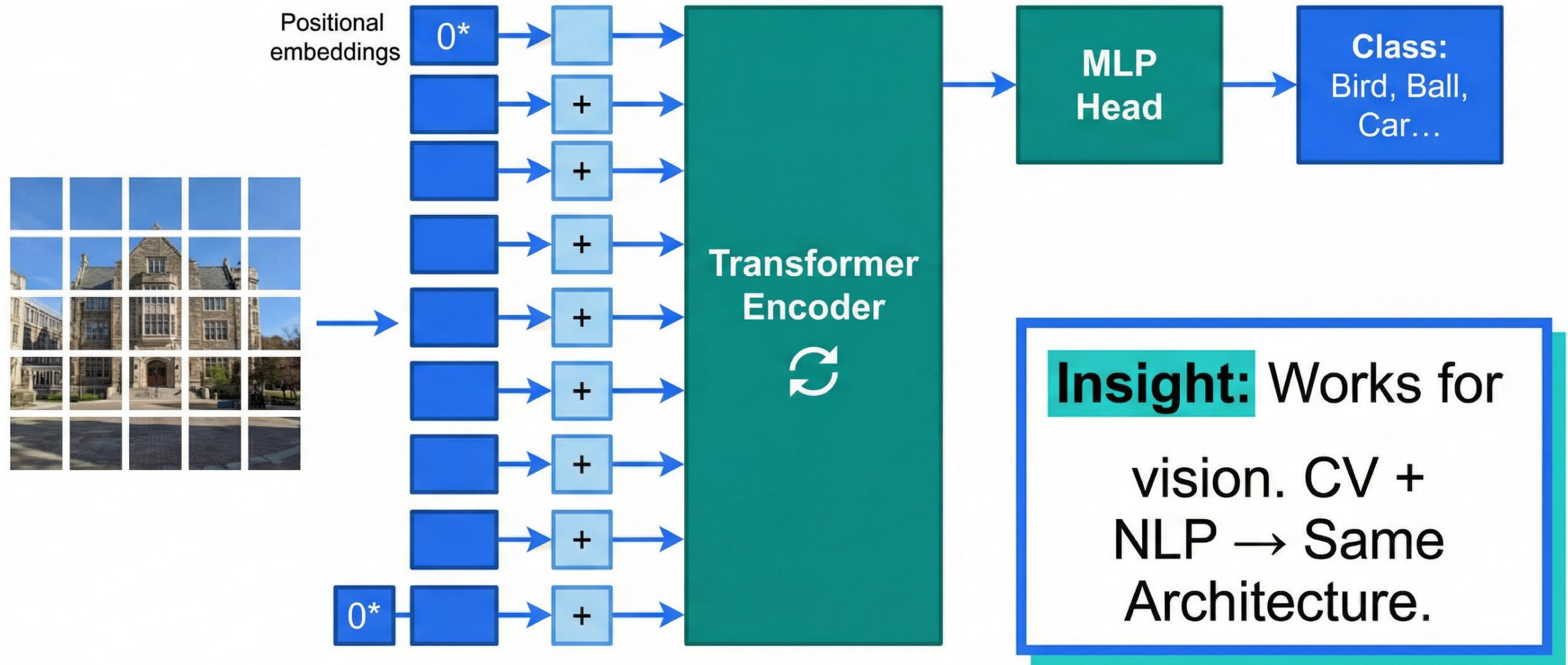
Act 2 Limitations

- ✗ One model, one task
- ✗ Understanding $\neq$ Generation
- ✗ Massive labeled data
- ✗ Lacks generalization

Can we unify them? 



Vision Transformer (2020)



Four Unifying Trends

1 Multimodal
Text + Vision

2 Zero-shot
No training

3 Visual Dialog
Reasoning

4 World Model
Unified



Image = Matrix



=

255	200	255
200	50	200
255	200	255

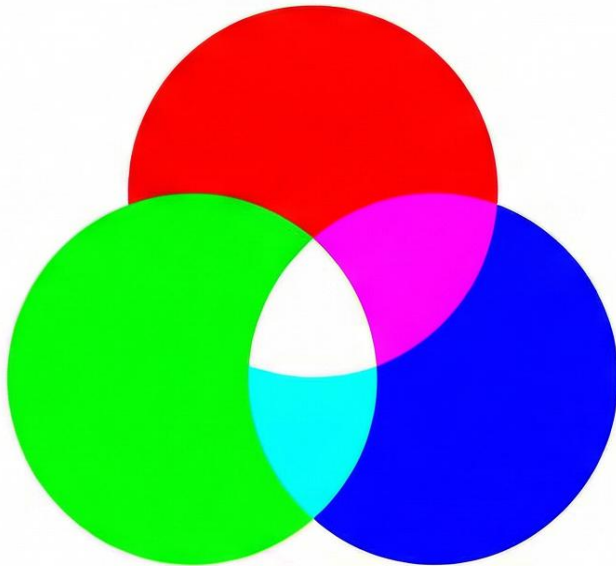
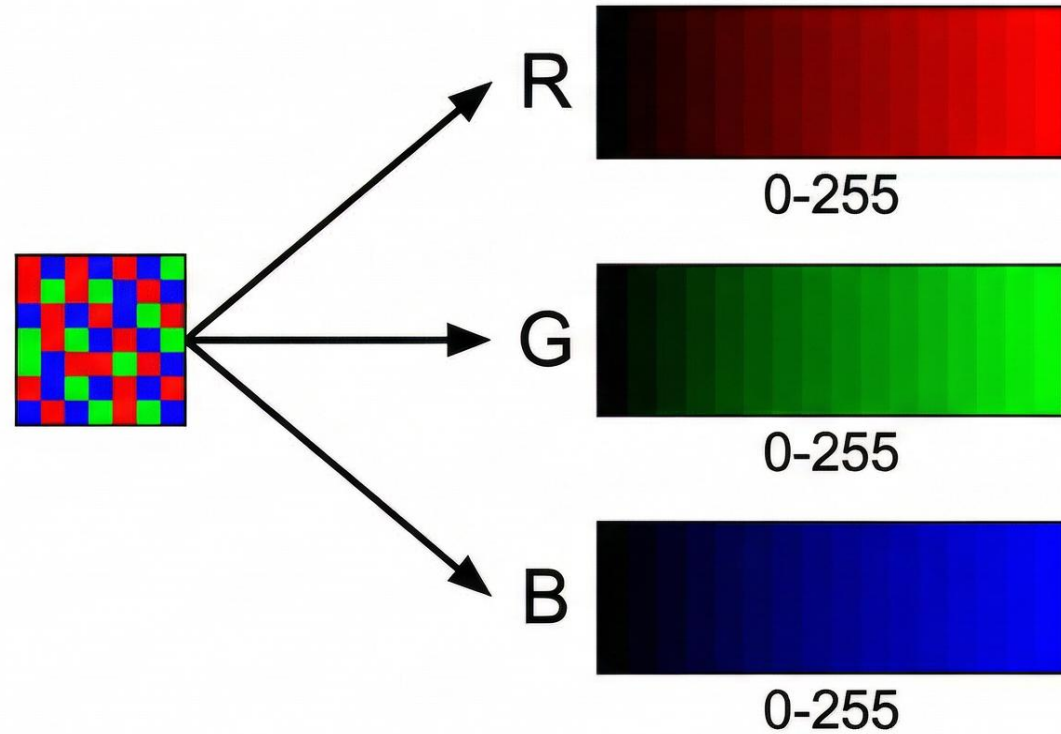
Each pixel = number
(grayscale) or 3
numbers (color)

Range: 0-255

Everything is
numbers.

Color = 3 Channels (RGB)

Color = 3 Channels
Each pixel = (R, G, B)
Each: 0-255



→ Red + Green + Blue → All colors

Quiz

为什么 **Transformer** 被认为是现代人工智能的基础模型？ 以下哪一项最能解释其成功的主要原因？

- A. 它天然支持文本、图像和音频等多模态输入。
- B. 相比先前的架构，它能更好地建模长程依赖关系。
- C. 随着数据、参数和算力的增加，它展现出强大且可预测的扩展行为。
- D. 其注意力机制与人类认知过程高度相似。

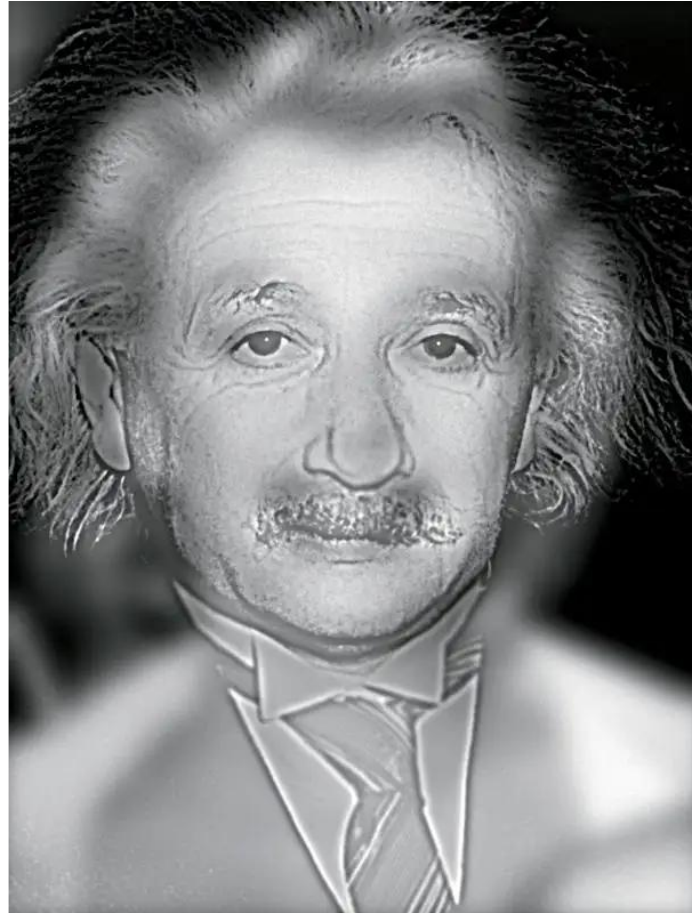


今日计划

图像滤波器及其应用！



这个人是谁？



图像滤波

- 图像滤波：在每个位置计算局部邻域的函数
- 非常重要！
 - 增强图像
 - 去噪、调整尺寸等。
 - 从图像中提取信息
 - 纹理、边缘、显著点等。
 - 检测模式
 - 模板匹配
 - 未来内容：卷积神经网络！



示例：盒式滤波器

$$\frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

“均值滤波器”：每个像素在邻域中都被同等重视。



图像滤波

$$[\cdot,\cdot]g \frac{1}{9}$$

1	1	1
1	1	1
1	1	1

$$[\cdot,\cdot]f$$

$$[\cdot,\cdot]$$

0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	0	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	0	0	0	0	0	0	0
0	0	90	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

$$[\cdot,\cdot]f = \sum_{k,l} f_{k,l} g_{k,l}$$

图像滤波

$$[\cdot, \cdot]_g \frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

$$[\cdot, \cdot]_f$$

$$[\cdot, \cdot]$$

0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	0	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	0	0	0	0	0	0	0
0	0	90	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

	0	10							

$$[\cdot, \cdot]_f = \sum_{k,l} [\cdot, \cdot]_g \cdot [\cdot, \cdot]_h + [\cdot, \cdot]_h$$

图像滤波

$$[\cdot, \cdot]_g = \frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

$[\cdot, \cdot]_f$

$[\cdot, \cdot]$

0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	0	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	0	0	0	0	0	0	0
0	0	90	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

	0	10	20						

$$[\cdot, \cdot]_f = \sum_{k,l} f_{kl} [\cdot, \cdot]_g + [\cdot, \cdot]$$



图像滤波

$$[\cdot, \cdot]_g = \frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

$[\cdot, \cdot]_f$

0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	0	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	0	0	0	0	0	0	0
0	0	90	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

$[\cdot, \cdot]$

	0	10	20	30					

$$[\cdot, \cdot]_f = \sum_{k,l} f_{k,l} [\cdot, \cdot]_g$$



图像滤波

$$[\cdot, \cdot]g = \frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

$[\cdot, \cdot]f$

0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	0	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	0	0	0	0	0	0	0
0	0	90	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

$[\cdot, \cdot]$

	0	10	20	30	30				

$$[\cdot, \cdot]f = \sum_{k,l} f_{kl} g_{kl} + [\cdot, \cdot]$$



图像滤波

$$[\cdot, \cdot]_g = \frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

$[\cdot, \cdot]_f$

$[\cdot, \cdot]$

0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	0	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	0	0	0	0	0	0	0
0	0	90	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

	0	10	20	30	30				
				?					

$$[\cdot, \cdot]_f = \sum_{k,l} f_{k,l} [\cdot, \cdot]_g$$



图像滤波

$$[\cdot, \cdot]_g = \frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

$[\cdot, \cdot]_f$

$[\cdot, \cdot]$

0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	0	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	0	0	0	0	0	0	0
0	0	90	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

	0	10	20	30	30				
						?			
				50					

$$[\cdot, \cdot]_f = \sum_{k,l} f_{k,l} g_{k,l} h_{k,l} + \cdot + \cdot$$



图像滤波

$$[\cdot, \cdot]_g = \frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

$$[\cdot, \cdot]_f$$

$$[\cdot, \cdot]$$

0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	0	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	0	0	0	0	0	0	0
0	0	90	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

	0	10	20	30	30	30	20	10	
	0	20	40	60	60	60	40	20	
	0	30	60	90	90	90	60	30	
	0	30	50	80	80	90	60	30	
	0	30	50	80	80	90	60	30	
	0	20	30	50	50	60	40	20	
	10	20	30	30	30	30	20	10	
	10	10	10	0	0	0	0	0	

$$[\cdot, \cdot]_f = \sum_{k,l} f_{k,l} [\cdot, \cdot]_g + [\cdot, \cdot]$$

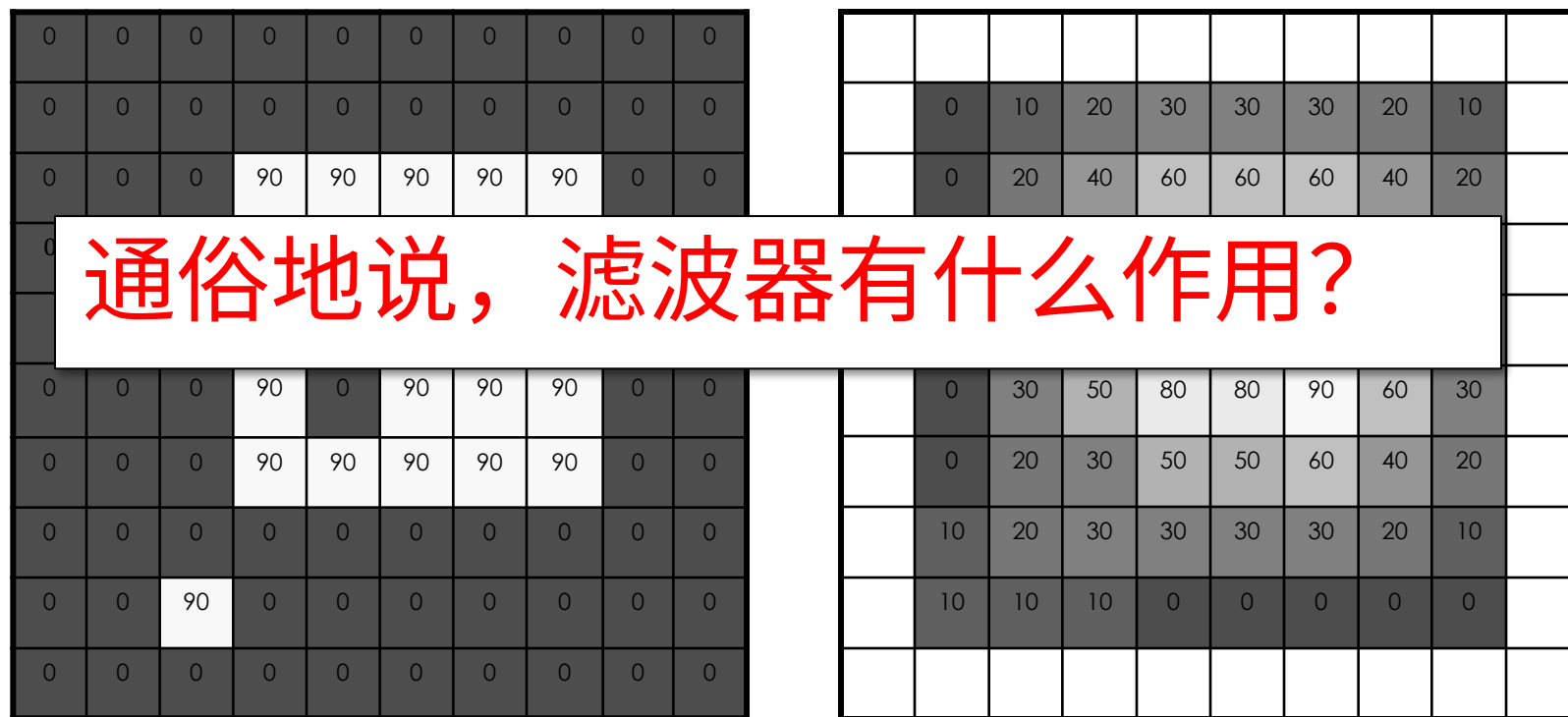


图像滤波

$$[\cdot, \cdot]_g = \frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

$[\cdot, \cdot]_f$

$[\cdot, \cdot]$



$$[\cdot, \cdot]_f = \sum_{k,l} f_{k,l} g_{k,l} h[\cdot, \cdot]$$

盒式滤波器

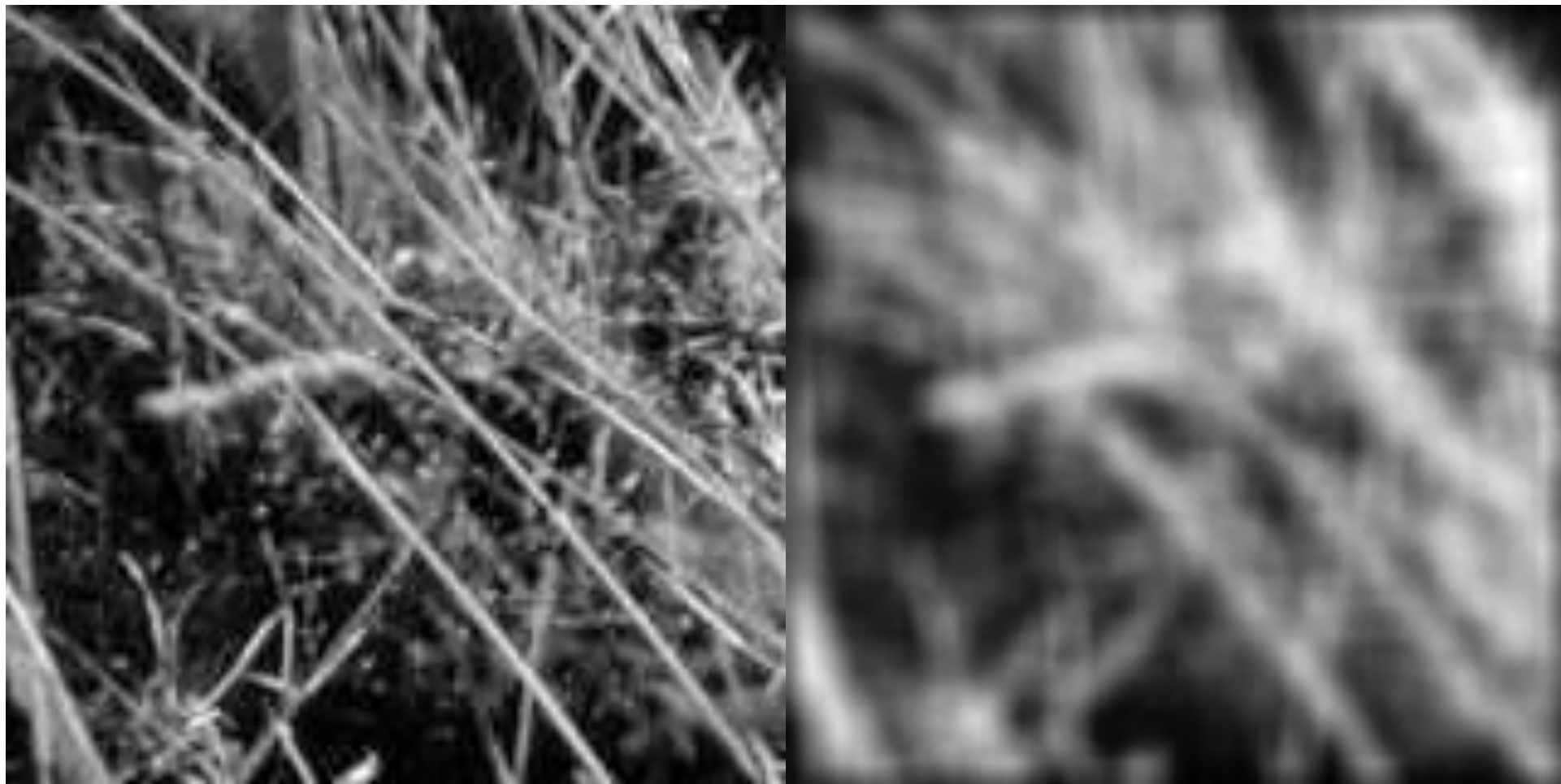
它有什么作用？

- 将每个像素替换为其邻域的平均值
- 实现平滑效果（去除尖锐特征）

$$\frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}g$$



使用盒式滤波器进行平滑



线性滤波器练习



原图

0	0	0
0	1	0
0	0	0

?

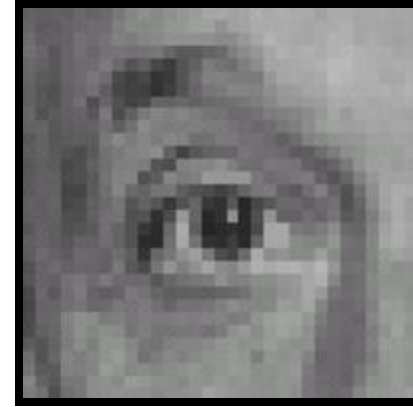


线性滤波器练习



原图

0	0	0
0	1	0
0	0	0



滤波后（无
变化）



线性滤波器练习



原始

0	0	0
0	0	1
0	0	0

?



线性滤波器练习



原图

0	0	0
0	0	1
0	0	0



向左平移
1 像素



线性滤波器练习



原图

0	0	0
0	2	0
0	0	0

—

$\frac{1}{9}$

1	1	1
1	1	1
1	1	1

?

(注意滤波器之和为 1)



线性滤波器练习



原图

0	0	0
0	2	0
0	0	0

—

$\frac{1}{9}$

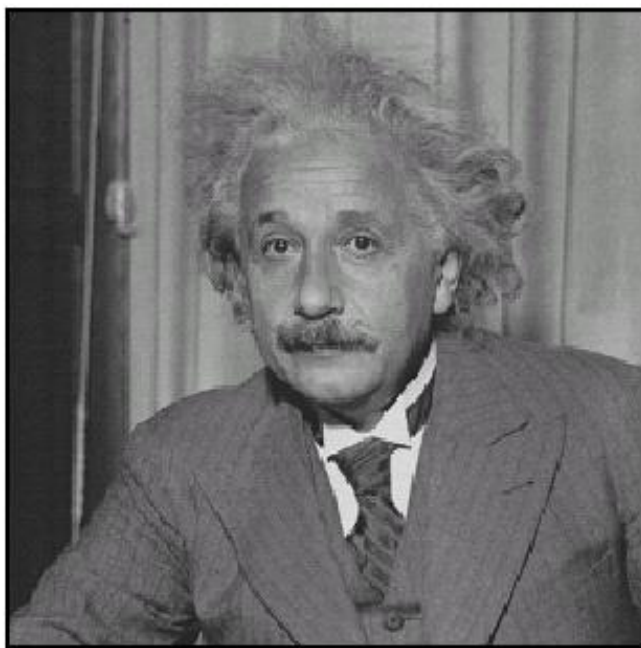
1	1	1
1	1	1
1	1	1



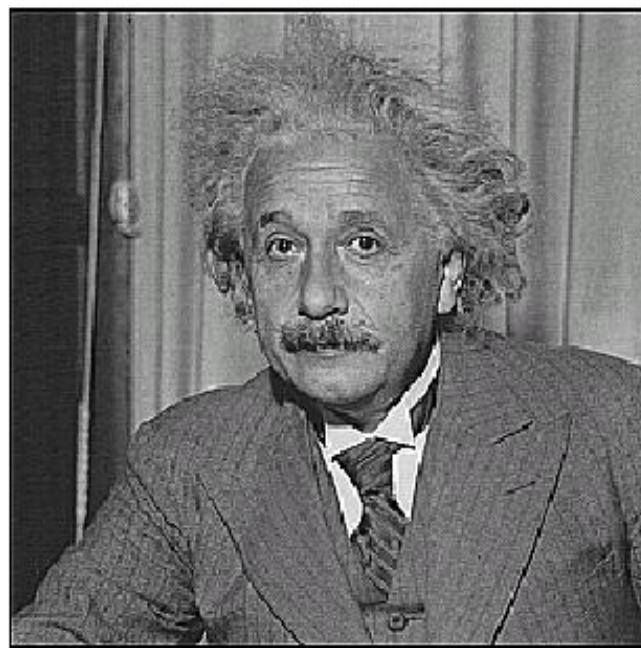
锐化滤波器 - 强调与局部平均值的差异



锐化



before



after



线性滤波器的关键性质

线性:

$$\text{filter}(f_1 + f_2) = \text{filter}(f_1) + \text{filter}(f_2)$$

平移不变性: 无论像素位置如何, 行为相同

$$\text{filter}(\text{shift}(f)) = \text{shift}(\text{filter}(f))$$

任何线性平移不变算子都可以表示为卷积



更多性质

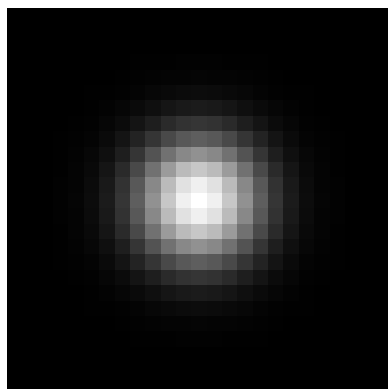
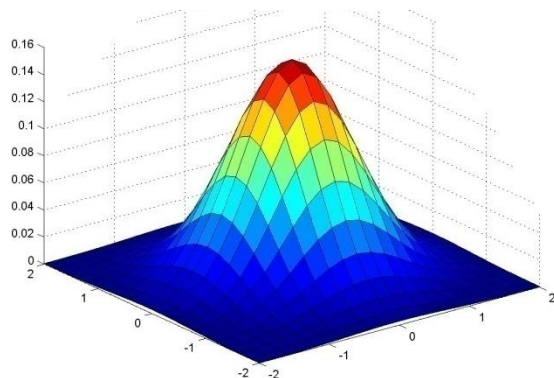
- 交换律: $a * b = b * a$
 - 从概念上讲, 滤波器与信号 (图像) 没有区别
- 结合律: $a * (b * c) = (a * b) * c$ • 通常依次应用多个滤波器:
 $((a * b_1) * b_2) * b_3$ • 这等价于应用一个滤波器: $a * (b_1 * b_2 * b_3)$
- 对加法满足分配律: $a * (b + c) = (a * b) + (a * c)$
- 标量可提取出来: $ka * b = a * kb = k(a * b)$
- 恒等性: 单位脉冲 $e = [0, 0, 1, 0, 0]$, $a * e = a$



重要滤波器：高斯

- 相邻像素基于邻近度的权重贡献

它使用加权平均，其中 **中心像素的影响最大**，并且周围像素的影响随着它们离中心越远而减小。



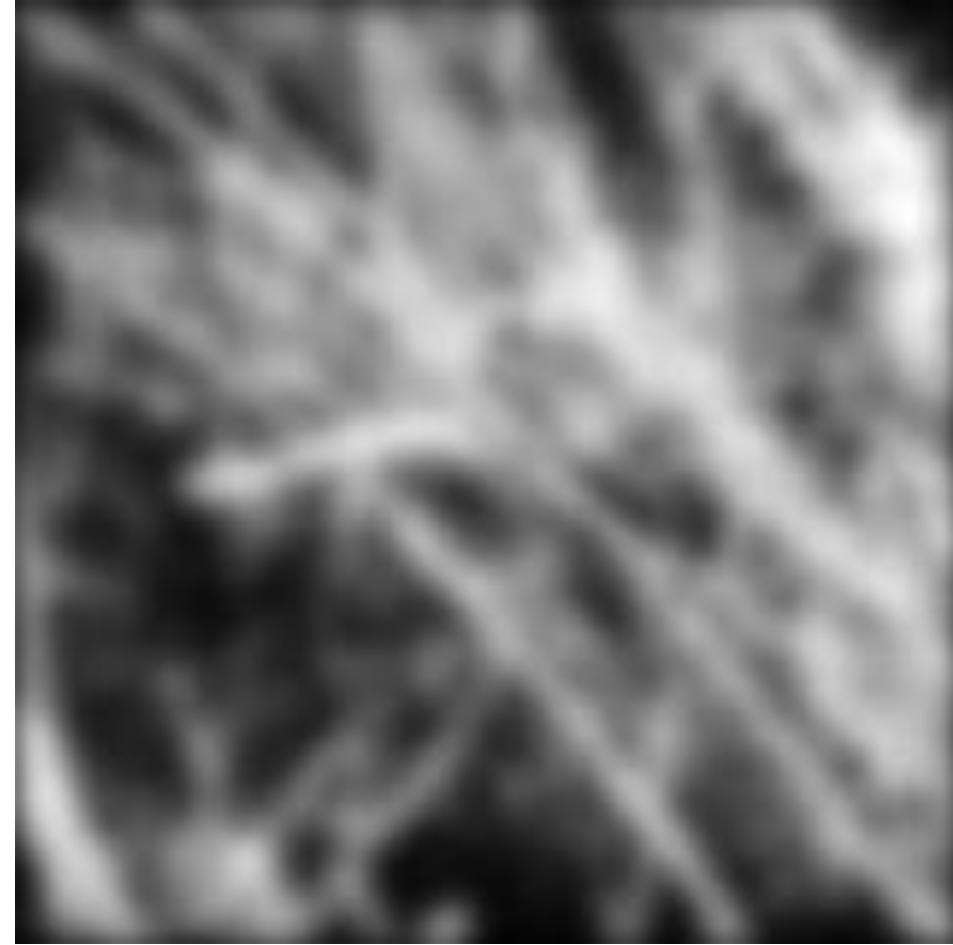
0.003	0.013	0.022	0.013	0.003
0.013	0.059	0.097	0.059	0.013
0.022	0.097	0.159	0.097	0.022
0.013	0.059	0.097	0.059	0.013
0.003	0.013	0.022	0.013	0.003

5 x 5, $\sigma = 1$

$$G_{\sigma} = \frac{1}{2\pi\sigma^2} e^{-\frac{(x^2+y^2)}{2\sigma^2}}$$



使用高斯滤波器进行平滑



高斯滤波器

- 从图像中移除“高-频”分量（低通滤波器）
 - 图像变得更平滑
- 与自身卷积得到另一个高斯函数– 因此可以用小宽度核进行平滑，重复操作，得到与较大宽度核相同的结果– 用宽度为 σ 的高斯核卷积两次，等同于用宽度为 $\sigma \sqrt{2}$ 的核卷积一次
- 可分离核
 - 可分解为两个一维高斯函数的乘积

更自然

更灵活

更高效



高斯滤波器的可分离性

$$\begin{aligned} G_{\sigma}(x, y) &= \frac{1}{2\pi\sigma^2} \exp^{-\frac{x^2 + y^2}{2\sigma^2}} \\ &= \left(\frac{1}{\sqrt{2\pi}\sigma} \exp^{-\frac{x^2}{2\sigma^2}} \right) \left(\frac{1}{\sqrt{2\pi}\sigma} \exp^{-\frac{y^2}{2\sigma^2}} \right) \end{aligned}$$

The 2D Gaussian can be expressed as the product of two functions, one a function of x and the other a function of y

In this case, the two functions are the (identical) 1D Gaussian



可分离性示例

$$O(M \cdot N \cdot m \cdot n)$$

二维滤波（仅中心位置）

1	2	1
2	4	2
1	2	1

 $\ast$

2	3	3
3	5	5
4	4	6

$$O(M \cdot N \cdot (m + n))$$

该滤波器可分解为一维滤波器的乘积：

1	2	1
2	4	2
1	2	1

 $=$

1
2
1

 $\times$

1	2	1
---	---	---

沿行执行滤波：

1	2	1
---	---	---

 $\ast$

2	3	3
3	5	5
4	4	6

 $=$

	11	
	18	
	18	

接着沿剩余的列进行滤波：

1
2
1

 $\ast$

	11	
	18	
	18	

 $=$

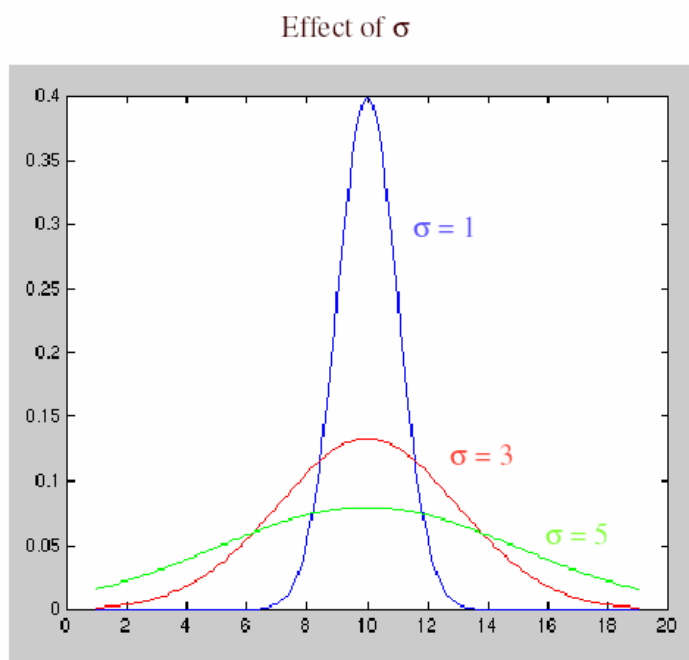
	65	



实际事项

滤波器应该多大？

- 边缘处的值应接近零
- 高斯核的经验法则：将核半宽设置为 $\geq 3\sigma$

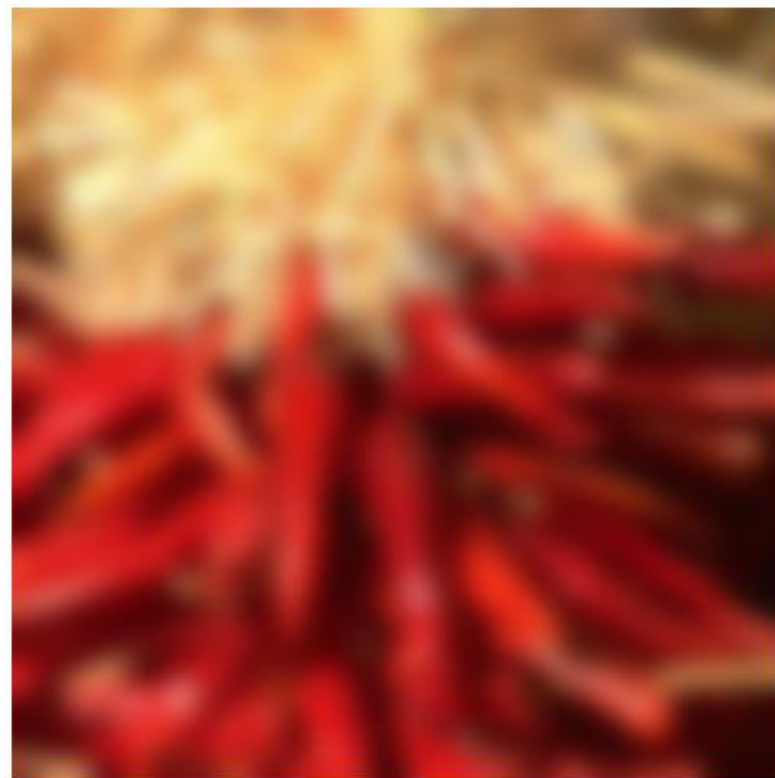


如果 $\sigma = 1$ ，你的卷积核半径应为 3，使得卷积核总大小为 7×7 （每侧 3 个像素 + 中心像素）。



实际问题

- 靠近边缘时怎么办呢？
 - 滤波窗口超出图像边缘
 - 需要进行外推
 - 方法：
 - 裁剪滤波器（黑色）：将图像边界外的所有像素视为零（黑色）。
 - 环绕：将图像视为环面，右边缘与左边缘相连。
 - 复制边缘：将边缘最后一个像素的值向外扩展。
 - 沿边缘反射：沿边界镜像图像像素以填充空白区域。



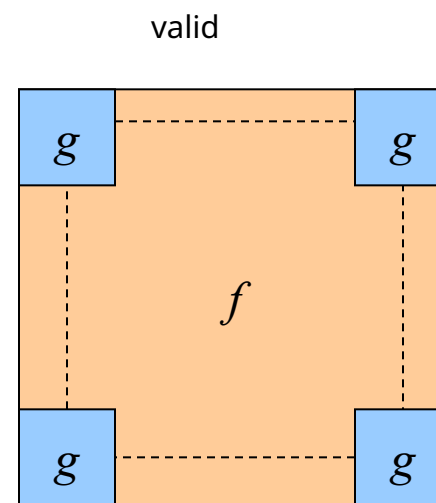
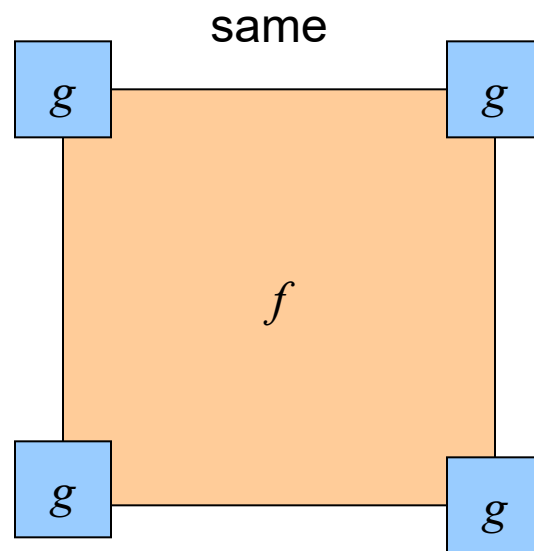
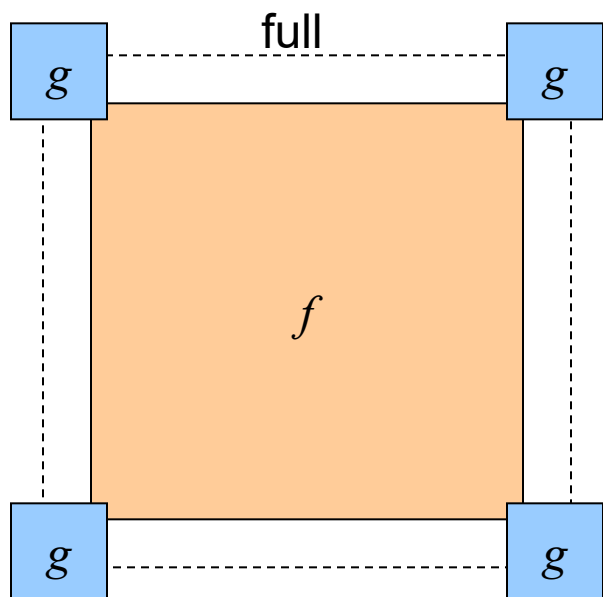
实际问题

- 方法（Python）：
 - 裁剪滤波器（黑色）：`convolve2d(f, g, boundary='fill', 0)`
 - 环绕：`convolve2d(f, g, boundary='wrap')`
 - 沿边缘反射：`convolve2d(f, g, boundary='symm')`

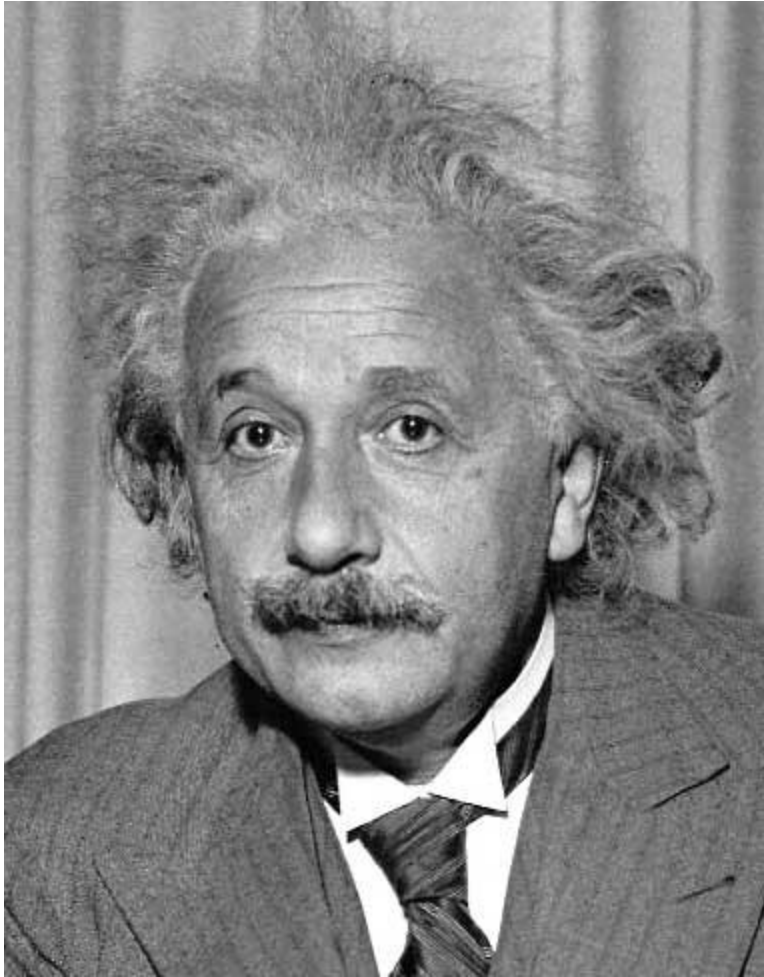


实际问题

- 输出的尺寸是多少？
- Python: `convolve2d(g, f, mode)`
 - $mode = \text{'full'}$: 输出大小为 f 和 g 的大小之和
 - $mode = \text{'same'}$: 输出大小与 f 相同
 - $mode = \text{'valid'}$: 输出大小为 f 和 g 的大小之差

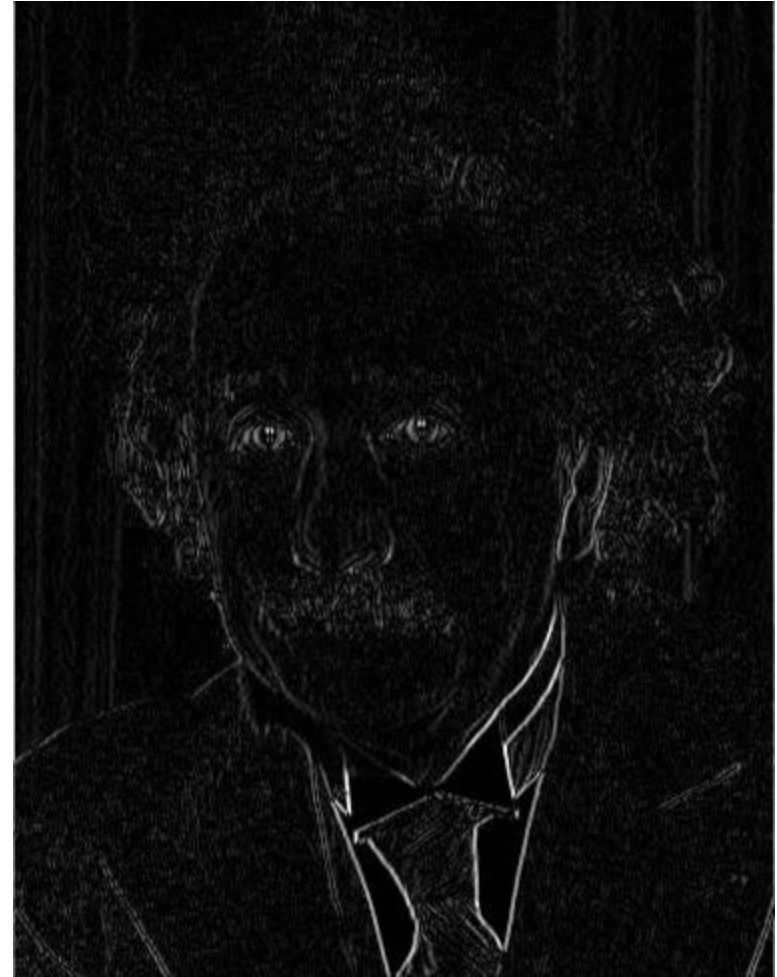


其他滤波器



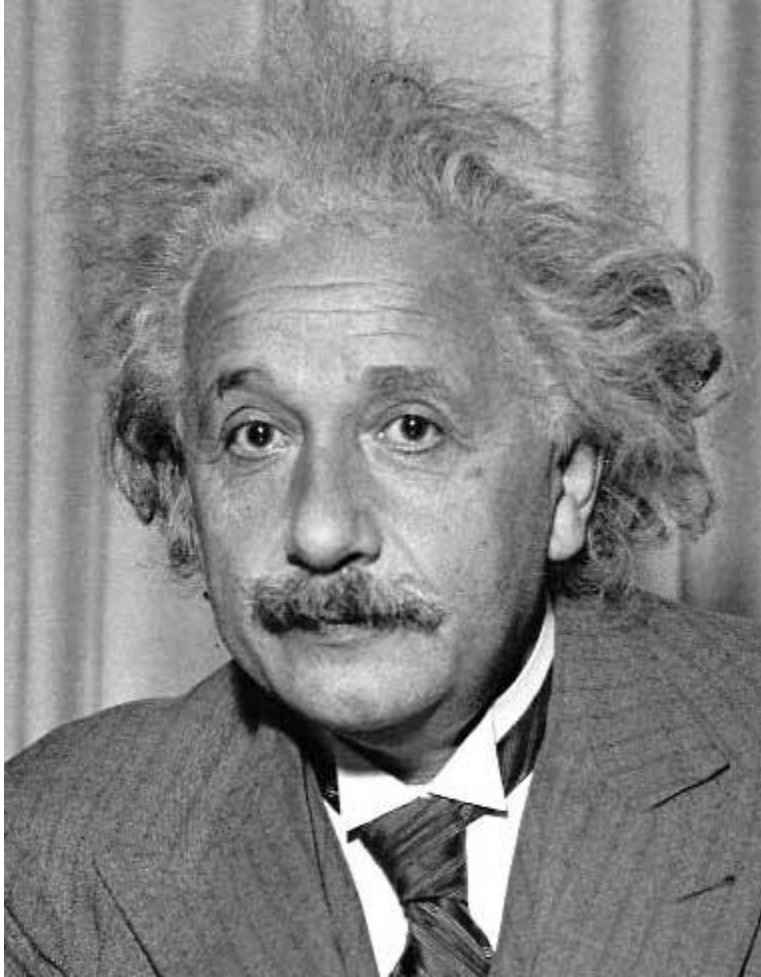
1	0	-1
2	0	-2
1	0	-1

Sobel



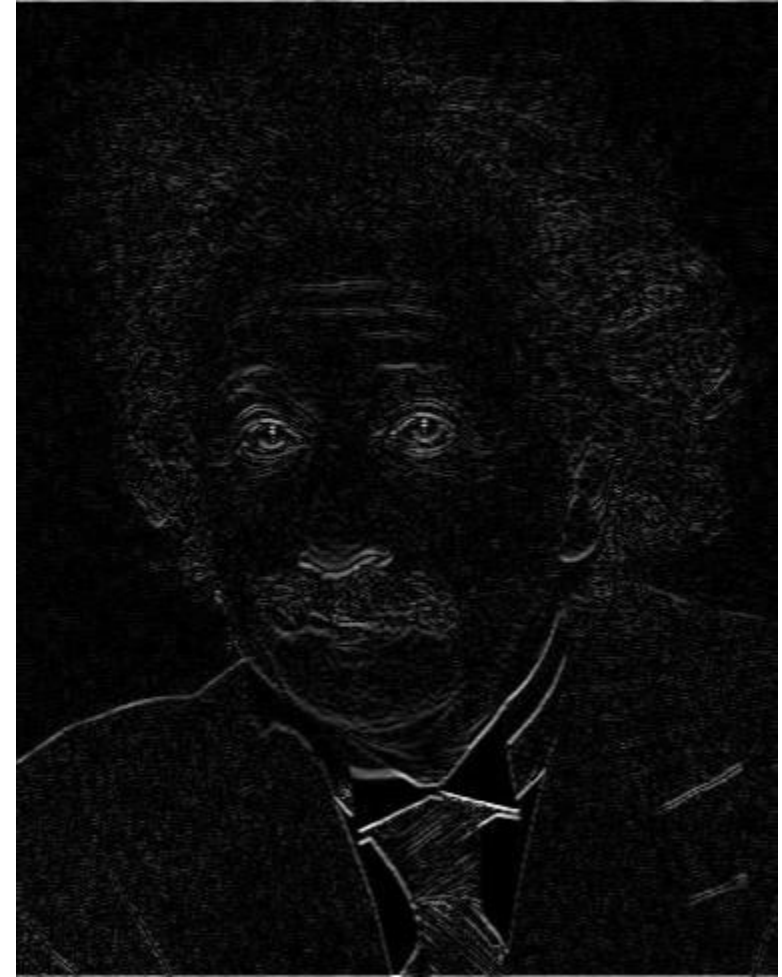
垂直边缘
(绝对值)

其他滤波器



1	2	1
0	0	0
-1	-2	-1

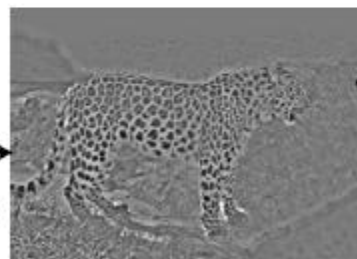
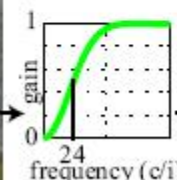
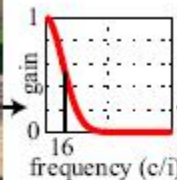
Sobel



水平边缘（绝对值）

应用：混合图像_A

高斯滤波器！



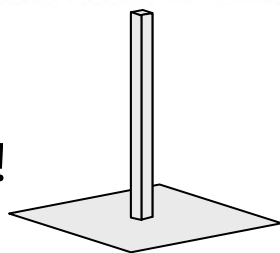
+



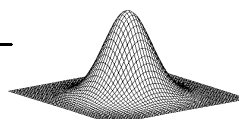
. Oliva, A. Torra
“混合图像,”

低频分量 + 高频分量

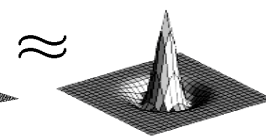
拉普拉斯滤波器！



单位冲激



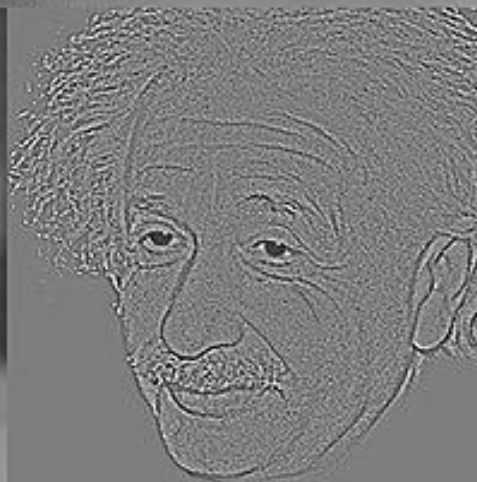
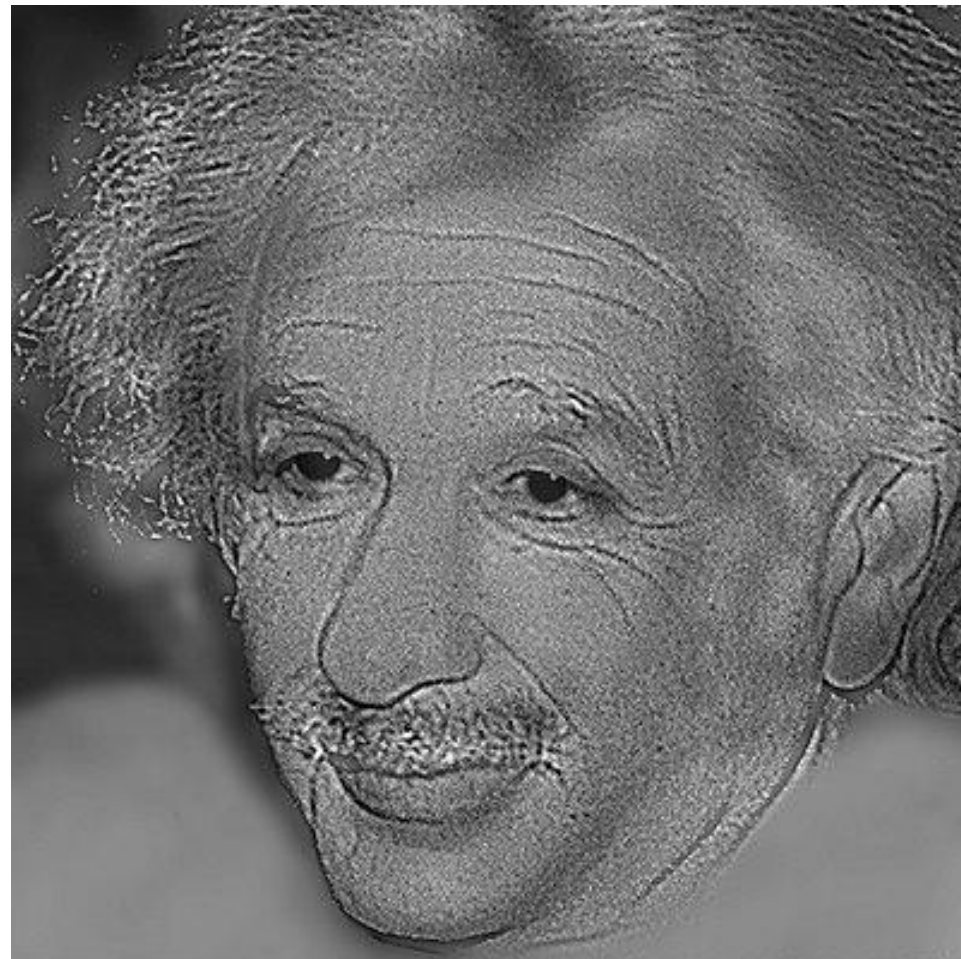
高斯



高斯拉普拉斯







https://en.wikipedia.org/wiki/Hybrid_image



其他滤波器应用

- 去噪
- 模板匹配
- 高斯金字塔
- 拉普拉斯金字塔

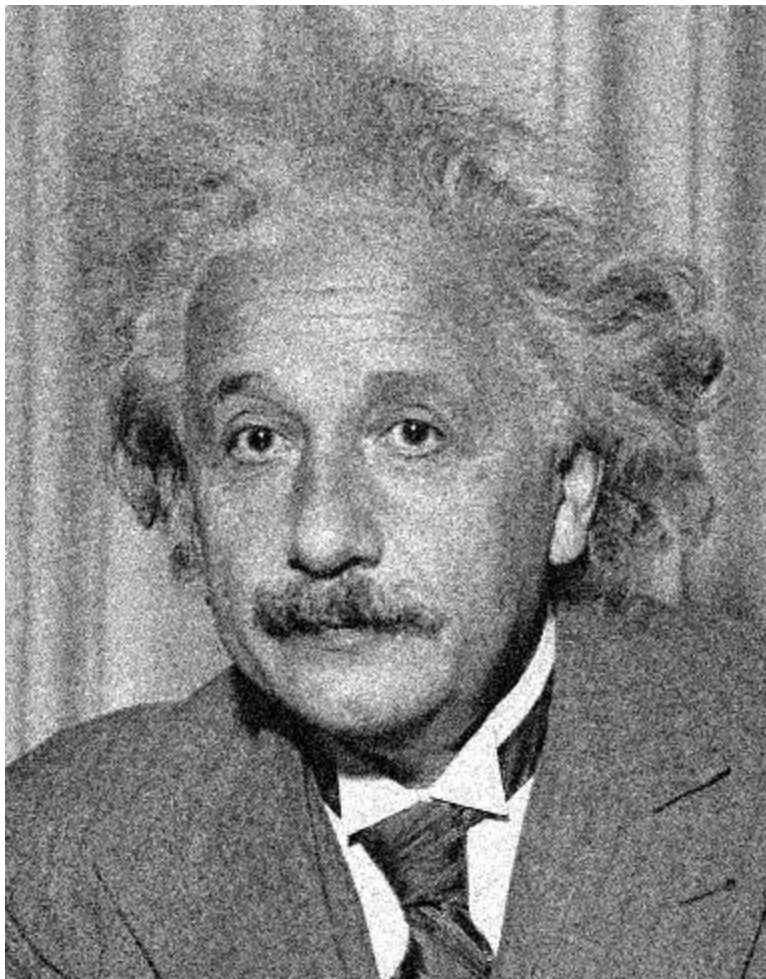


其他滤波器应用

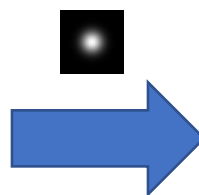
- 去噪
- 模板匹配
- 高斯金字塔
- 拉普拉斯金字塔



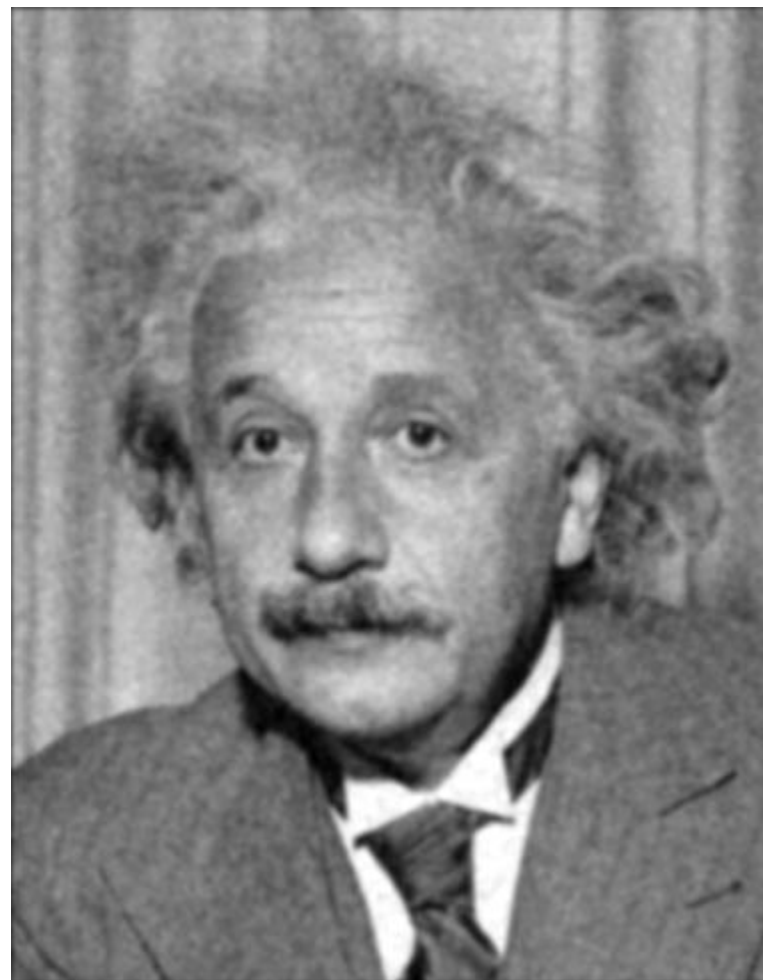
去噪



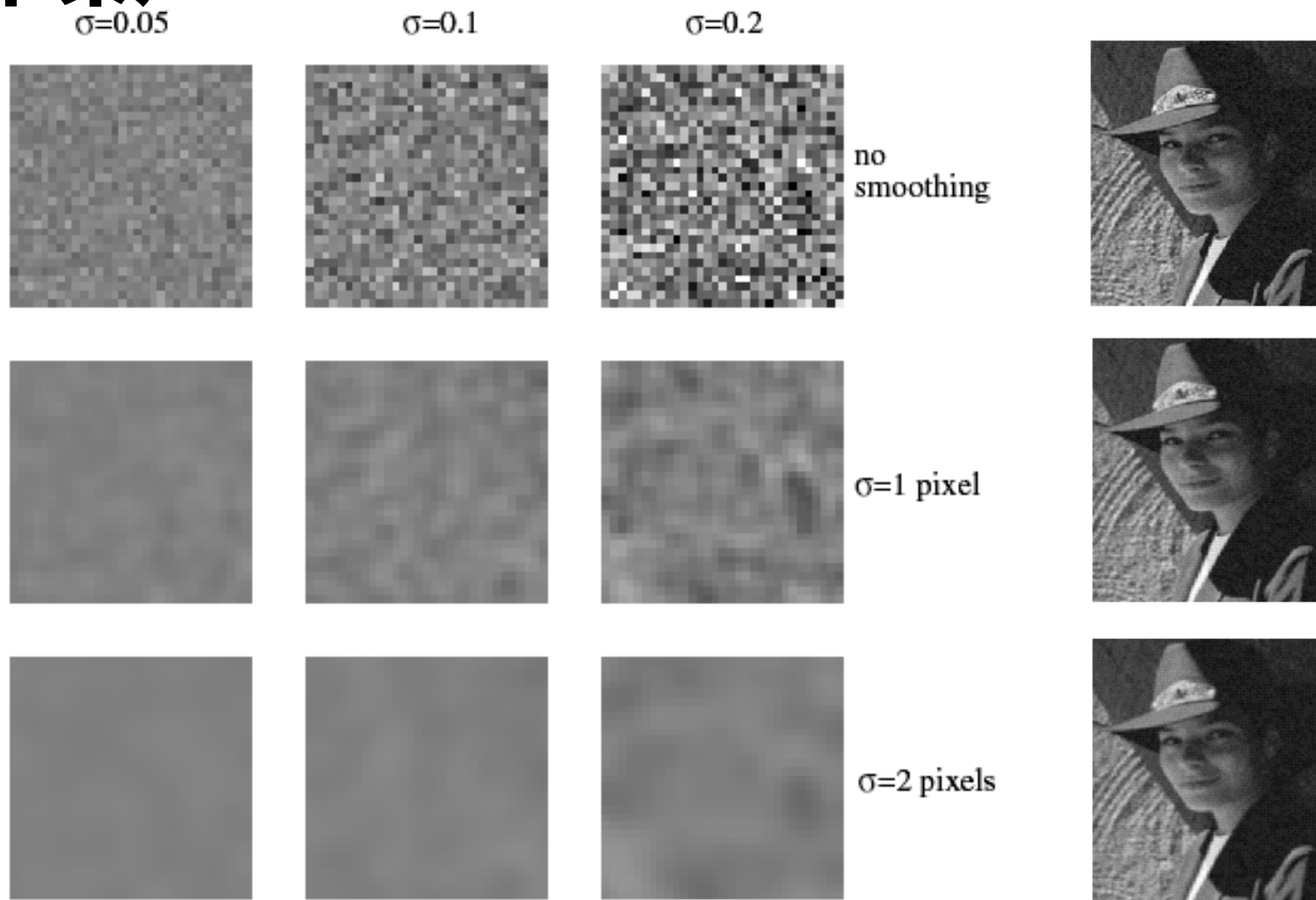
加性高斯噪声



高斯滤波器



降低高斯噪声



使用较大标准差进行平滑可以抑制噪声，但也会模糊图像



通过高斯平滑降低椒盐噪声（脉冲噪声）

3x3



5x5

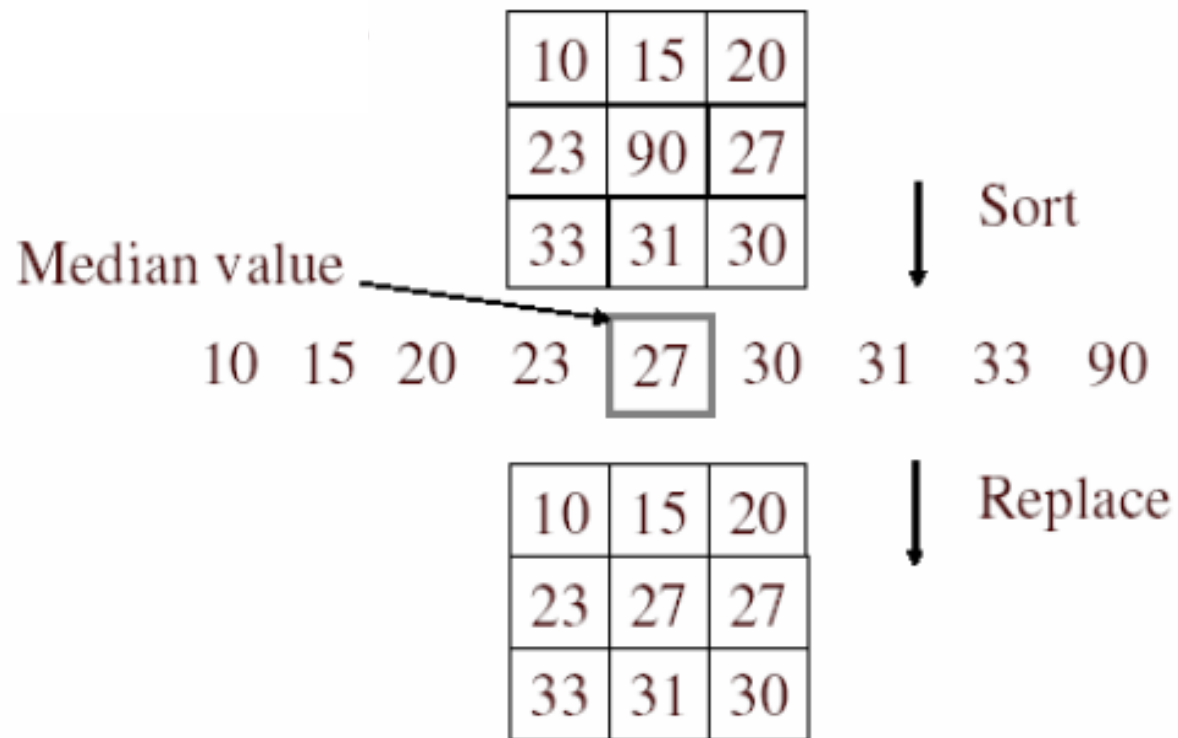


7x7



替代思路：中值滤波

- 中值滤波器通过选择窗口内的中值强度来对窗口进行操作



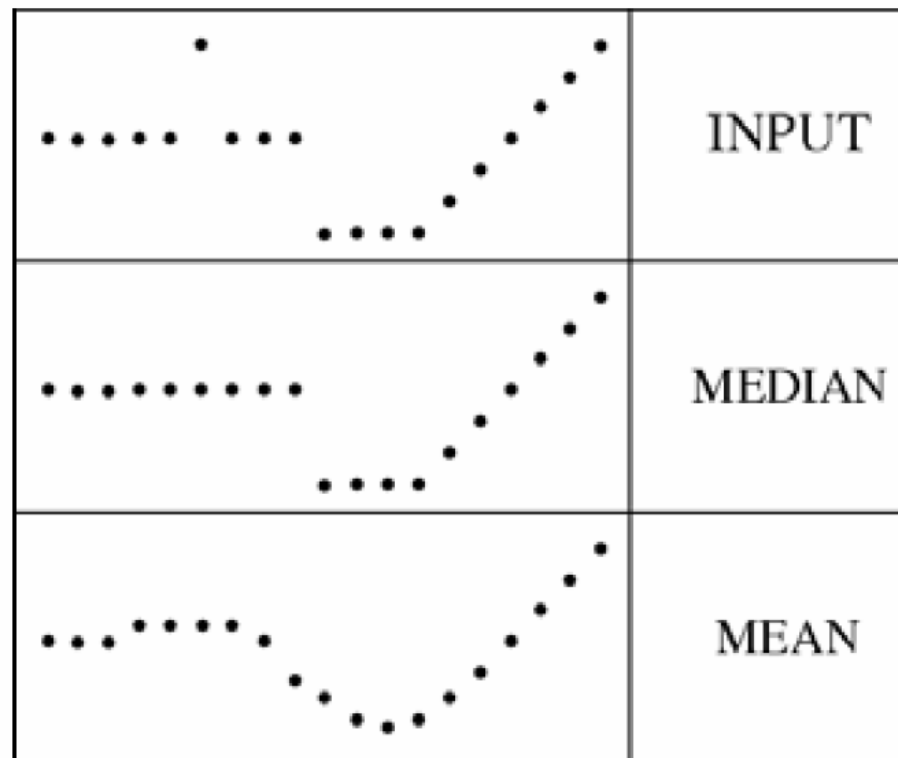
- 中值滤波是线性的吗？



中值滤波

- 中值滤波相比高斯滤波有什么优势?
- 对离群值的鲁棒性

filters have width 5 :

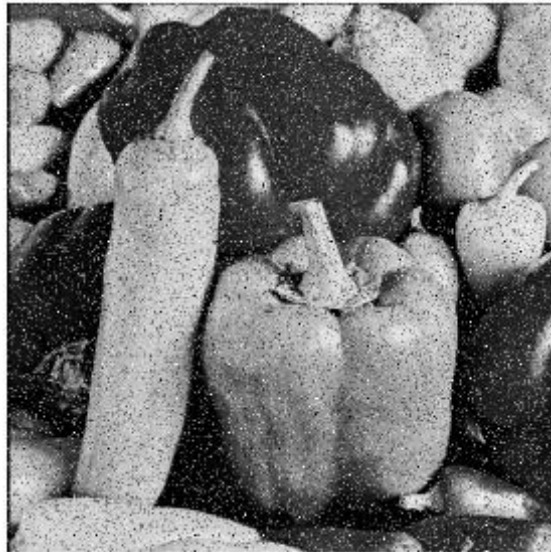


来源: K. Grauman

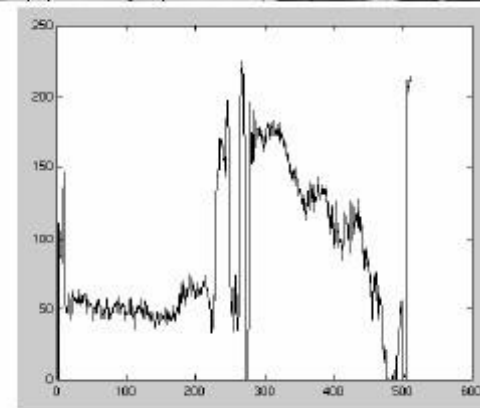
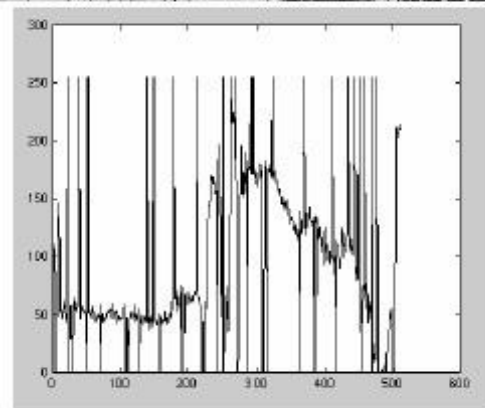


中值滤波

椒盐噪声



中值滤波后



Python: `scipy.ndimage.median_filter(image, size)`



中值滤波示例



original image



1px median filter



3px median filter



10px median filter

<http://en.wikipedia.org/wiki/File:Medianfilterp.png>

http://en.wikipedia.org/wiki/File:Median_filter_example.jpg



中值滤波与高斯滤波

3x3

5x5 7x7

高斯



中值



其他滤波器选择

- 加权中值（离中心越远的像素权重越小）
- 截尾均值（忽略少数最亮和最暗像素后取平均）
- 双边滤波（根据空间距离和强度差异进行加权）

`cv2.bilateralFilter(size, sigma color, sigma spatial)`



双边滤波

$$BF[I]_p = \frac{1}{W_p} \sum_{q \in S} G_{\sigma_s}(\|p - q\|) G_{\sigma_r}(|I_p - I_q|) I_q$$

Diagram illustrating the Bilateral Filter equation components:

- $\frac{1}{W_p}$: Normalization Factor
- $G_{\sigma_s}(\|p - q\|)$: Space Weight (represented by a 2D Gaussian heatmap)
- $G_{\sigma_r}(|I_p - I_q|)$: Range Weight (represented by a 1D Gaussian curve)



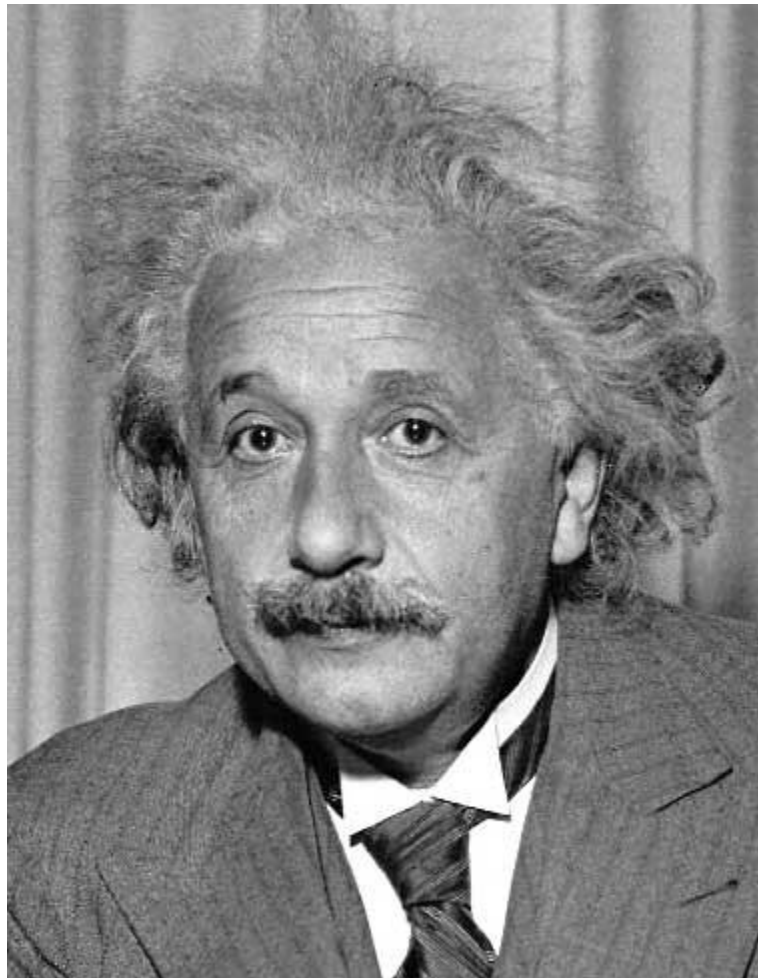
大纲：滤波的一些应用

- 去噪
- 模板匹配
- 高斯金字塔
- 拉普拉斯金字塔



模板匹配

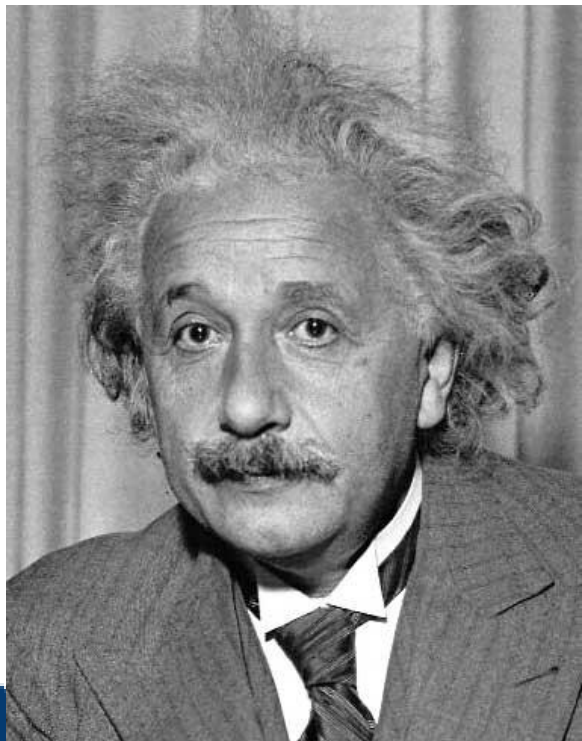
- 目标：在图像中查找
- 主要挑战：两个图像块之间什么是良好的相似度或距离度量？



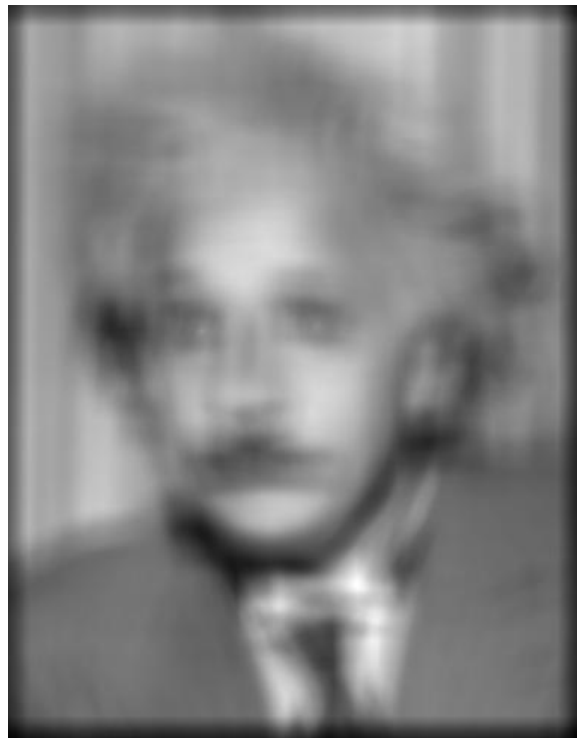
使用滤波器进行匹配

- 目标：在👁️图像中查找
- 方法 0：使用眼睛图像块对图像进行滤波

$$h[m,n] = \sum_{k,l} f[k,l] \text{ im}[m+k, n+l]$$



输入



滤波后的图像

亮度偏置：一般明亮的区域会产生高响应，无论它们看起来是否像眼睛，因为大数相乘会得到很大的和。这就是为什么它看起来像原始人脸的一幅明亮而模糊的版本。

im = 图
像 f = 滤波器

出了什么问题？



使用滤波器进行匹配

- 目标：在图像中查找
- 方法1：用以下滤波器对图像进行滤波：零均值眼睛

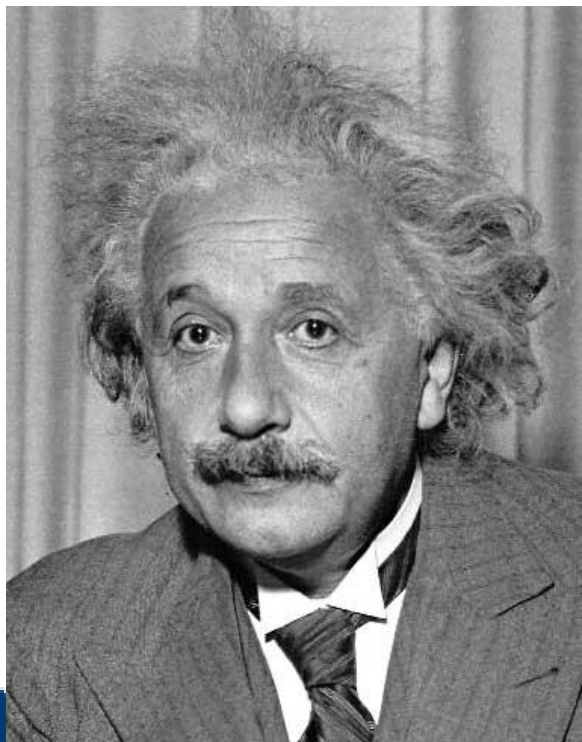
$$h[m,n] = \sum_{k,l} (f[k,l] - \bar{f}) im[m+k, n+l]$$

← mean 滤波器 f 的

• **模式优先于亮度**：通过减去平均强度 ($\bar{f}$)，滤波器只有在明暗的相对模式匹配时才会给出强烈的正响应。

• **消除均匀区域**：在完全平坦且明亮的区域（例如额头）中，零均值滤波器的求和结果为零，从而实际上忽略该区域。

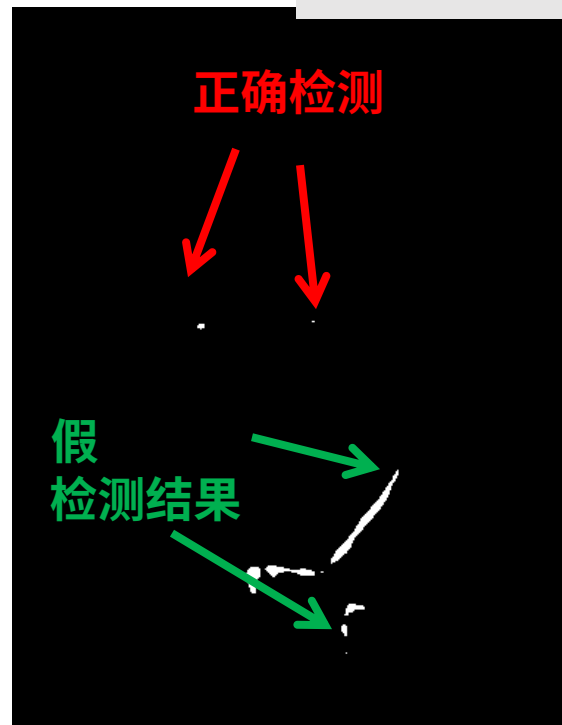
• **真实检测**：这使“阈值化图像”能够在眼睛位置精确显示清晰的小点，而不是在整个面部呈现发光斑块。



输入



滤波后图像（缩放）



阈值化图像

正确检测

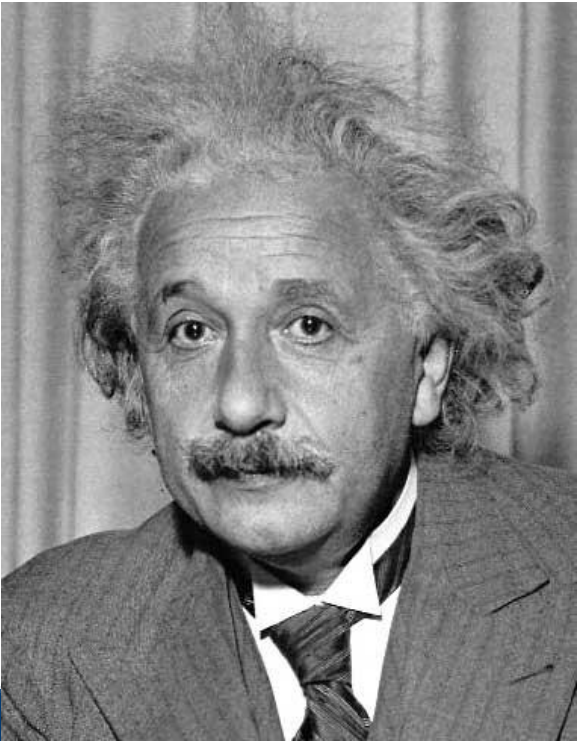
假
检测结果



基于滤波器的匹配

- 目标：在图像中查找
- 方法2：平方距离和 (SSD)

$$h[m,n] = \sum_{k,l} (f[k,l] - im[m+k,n+l])^2$$



输入



1- sqrt(SSD)



阈值化图像



使用滤波器进行匹配

SSD 是否可以用线性滤波器实现?

滤波要快得多!

$$h[m,n] = \sum_{k,l} (f[k,l] - im[m+k,n+l])^2$$

$$h[m,n] = \sum_{k,l} (f[k,l]^2 - 2 \cdot im[m+k,n+l] \cdot f[k,l] + im[m+k,n+l]^2)$$

$$h[m,n] = \sum_{k,l} f[k,l]^2 - 2 \sum_{k,l} im[m+k,n+l] \cdot f[k,l] + \sum_{k,l} im[m+k,n+l]^2$$

$$h = \sum_{k,l} f[k,l]^2 - 2 \text{filter}(im, f) + \text{filter}(im.^2, \text{ones}(f.\text{shape}))$$

常数

线性滤波器

逐元素平方 f, 然后求和
使用大小为 f 的全 1 核



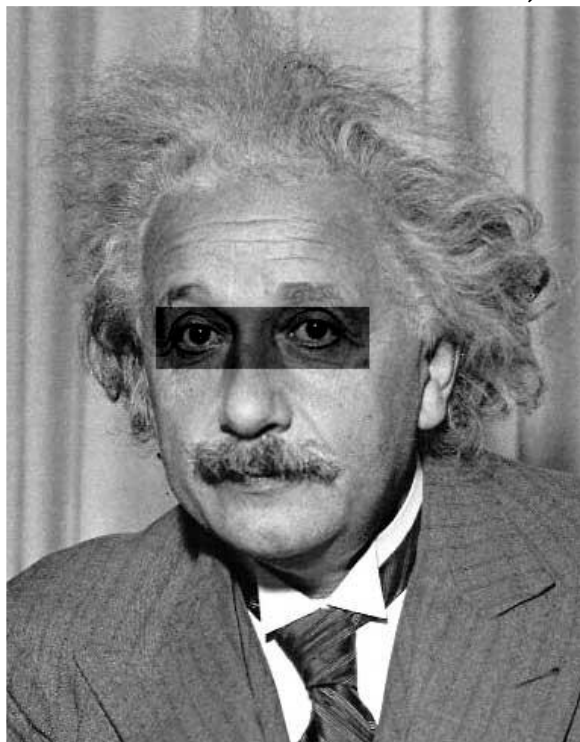
使用滤波器进行匹配

- 目标：在图👁象中查找
- 方法2：平方距离和（SSD）

$$h[m,n] = \sum_{k,l} (f[k,l] - im[m+k,n+l])^2$$

SSD 的潜在缺点是什么？

对强度偏移敏感



输入



1- sqrt(SSD)



使用滤波器进行匹配

- 目标：在像中查找
- 方法3：归一化互相关

$$h[m, n] = \frac{\sum (f - \bar{f})(im - \overline{im})}{\sqrt{\sum (f - \bar{f})^2 \sum (im - \overline{im})^2}}$$

余弦相似度!

$$h[m, n] = \frac{\begin{array}{c} \text{模板均值} \\ \downarrow \\ \sum_{k,l} (f[k, l] - \bar{f}) \end{array} \begin{array}{c} \text{图像块均值} \\ \downarrow \\ \sum_{k,l} (im[m + k, n + l] - \overline{im_{m,n}}) \end{array}}{\left(\sum_{k,l} (f[k, l] - \bar{f})^2 \sum_{k,l} (im[m + k, n + l] - \overline{im_{m,n}})^2 \right)^{0.5}}$$

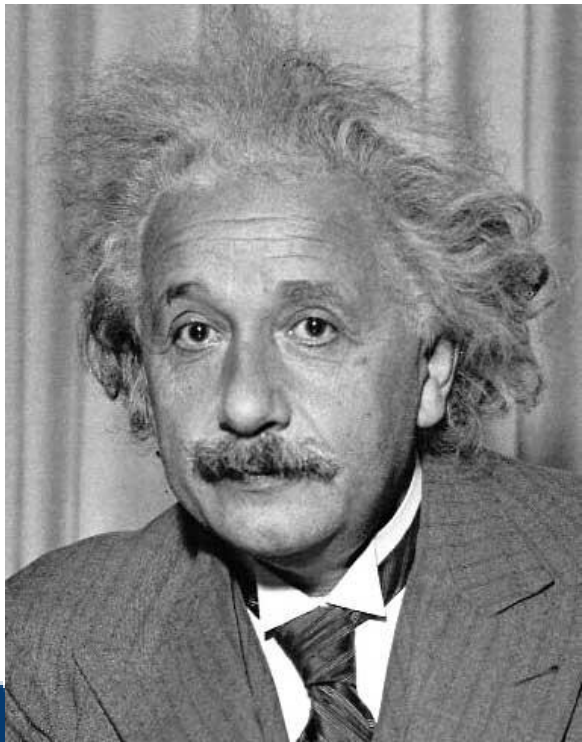
(除以模板与图像块的标准差之积)

Python: `cv2.matchTemplate(im, template, cv2.TM_CCOEFF_NORMED)`



使用滤波器进行匹配

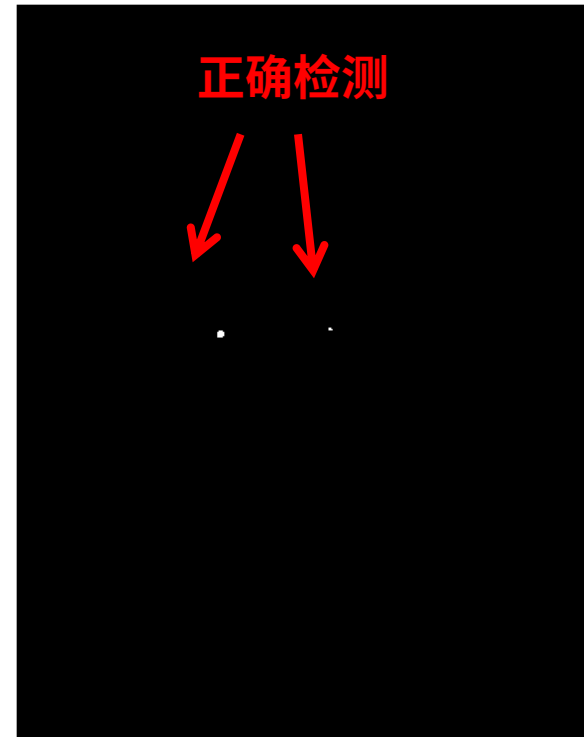
- 目标：在图像中查找
- 方法3：归一化互相关



输入



归一化互相关

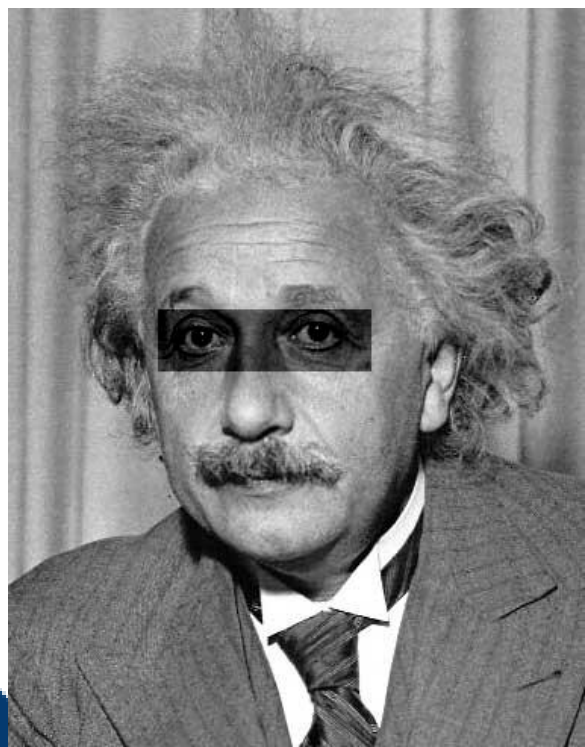


阈值化图像



基于滤波器的匹配

- 目标：在图像中查找
- 方法 3：归一化互相关



输入



归一化互相关



阈值化图像

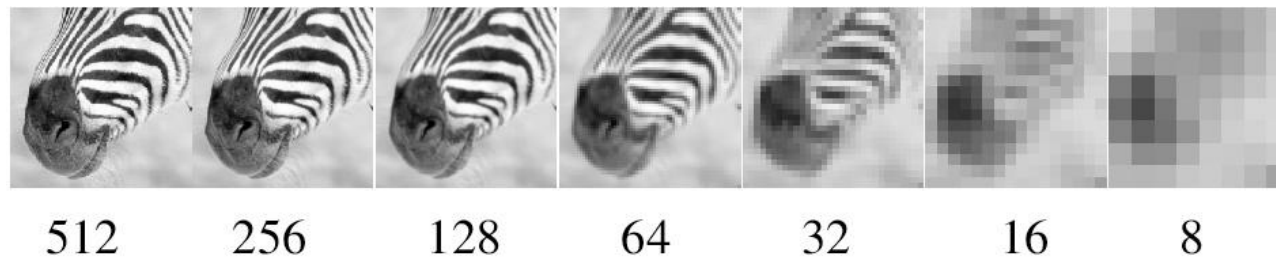


问：如果我们想找到更大或更小的眼睛怎么办？

A：图像金字塔



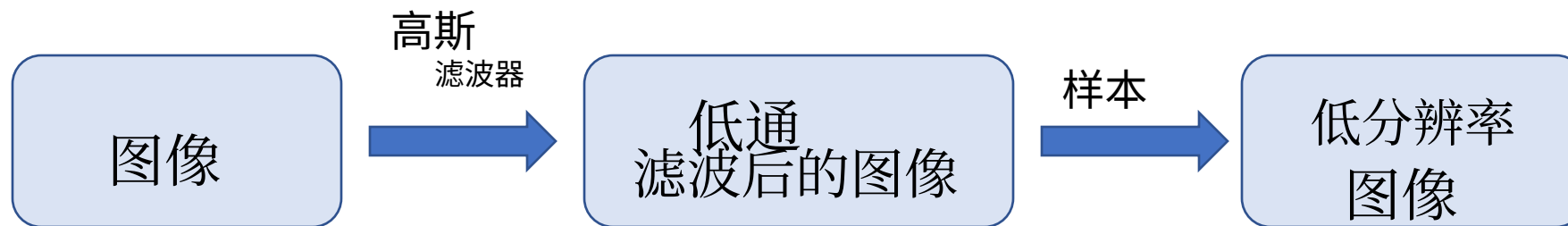
高斯金字塔



来源: Forsyth



采样回顾



为什么需要低通滤波器？



高分辨率图像



直接下采样



使用低通滤波器



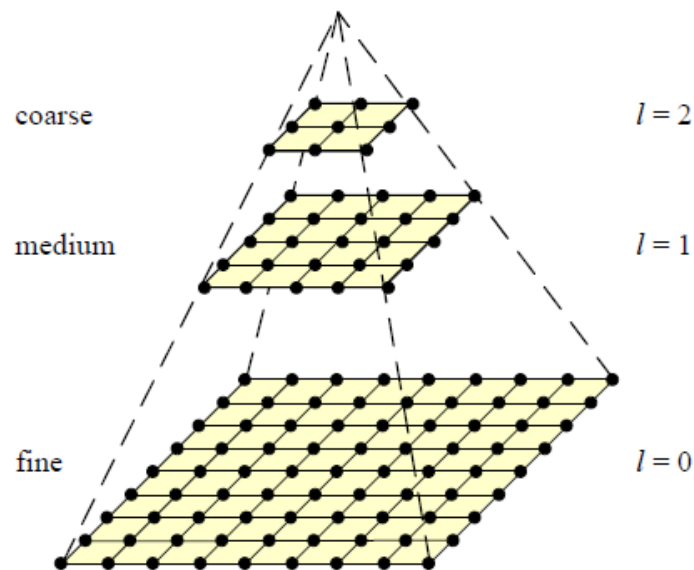
还有高斯金字塔的其他优势吗？

加速

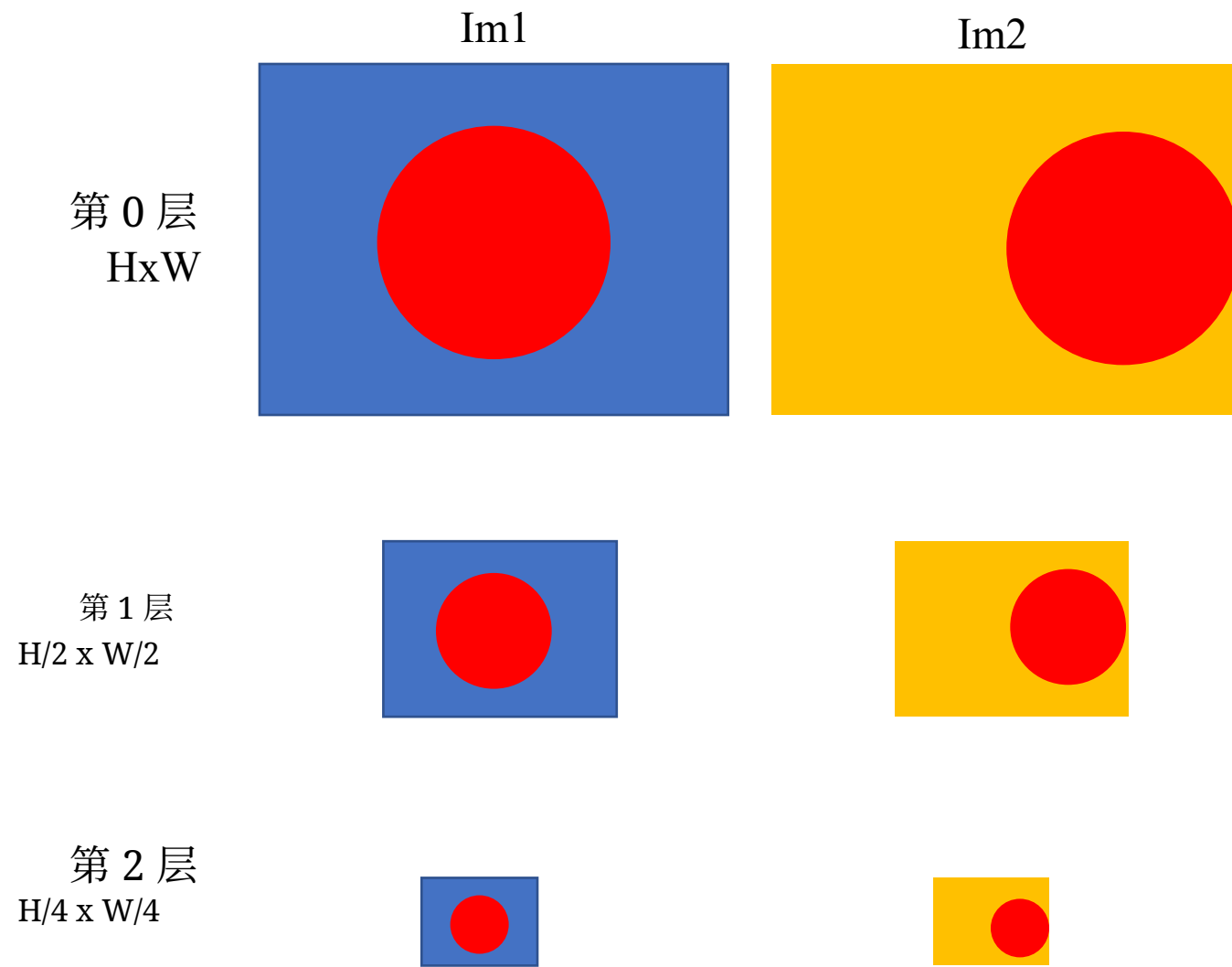


由粗到细的图像配准

1. 计算高斯金字塔
2. 在粗层级进行对齐
 - 找到最小 SSD 位置。
3. 在更细的层级上依次对齐
 - 在以较粗尺度确定的位置为中心的小范围内搜索（例如 5x5）

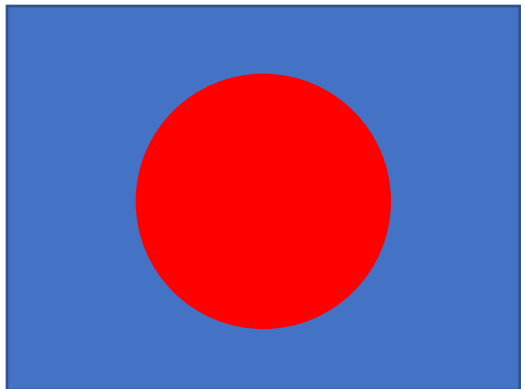


由粗到细的图像配准

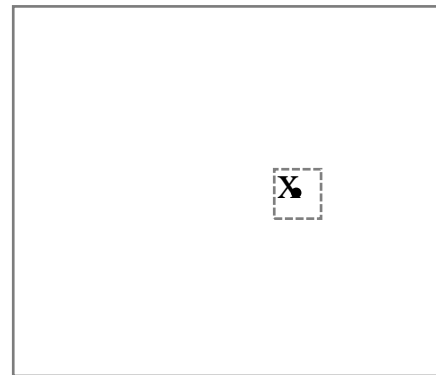
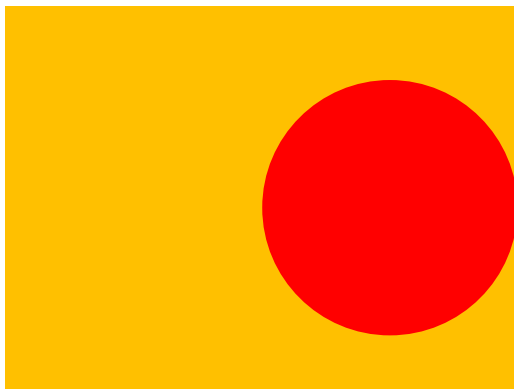


由粗到精的图像配准

Im1

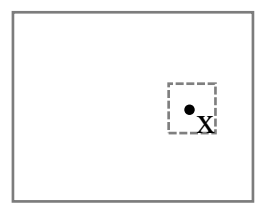
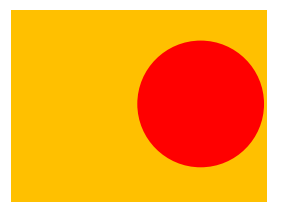
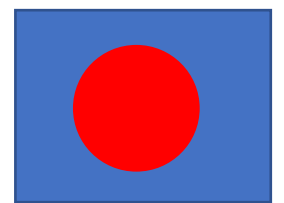


Im2



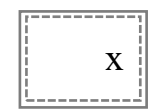
$$tx_0, ty_0 = \operatorname{argmin}_{tx \in 2 \cdot tx_1 + \{-s..s\}, ty \in 2 \cdot ty_1 + \{-s..s\}}$$

$$\text{SSD}(im1_0, \text{translate } im2_0, tx, ty)$$



$$tx_1, ty_1 = \operatorname{argmin}_{tx \in 2 \cdot tx_2 + \{-s..s\}, ty \in 2 \cdot ty_2 + \{-s..s\}}$$

$$\text{SSD}(im1_1, \text{translate } im2_1, tx, ty)$$

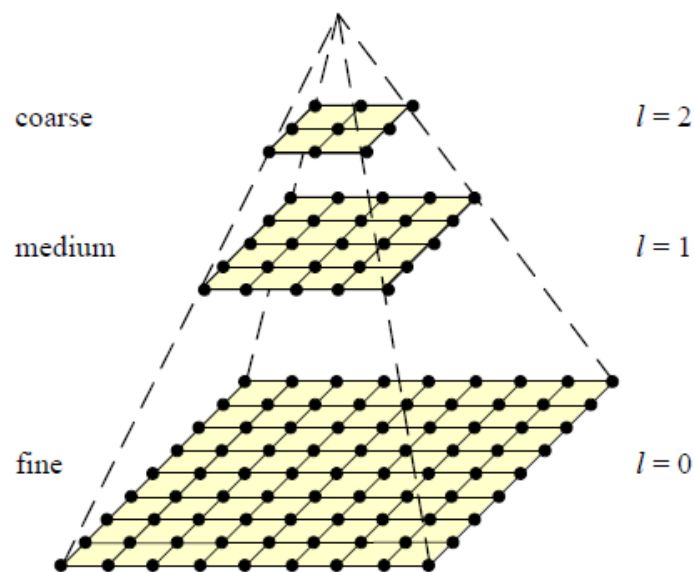


$$tx_2, ty_2 = \operatorname{argmin}_{tx \in -\frac{W}{8}.. \frac{W}{8}, ty \in \{-\frac{H}{8}.. \frac{H}{8}\}} \text{SSD}(im1_2, \text{translate } im2_2, tx, ty)$$



粗到细图像配准

1. 计算高斯金字塔
2. 在粗层级进行对齐
 - 寻找 SSD 最小的位置
3. 依次在更精细层级进行对齐
 - 在以较粗尺度确定的位置为中心的小范围内搜索（例如 5x5）



为什么这样会更快？

我们能保证得到相同的结果吗？

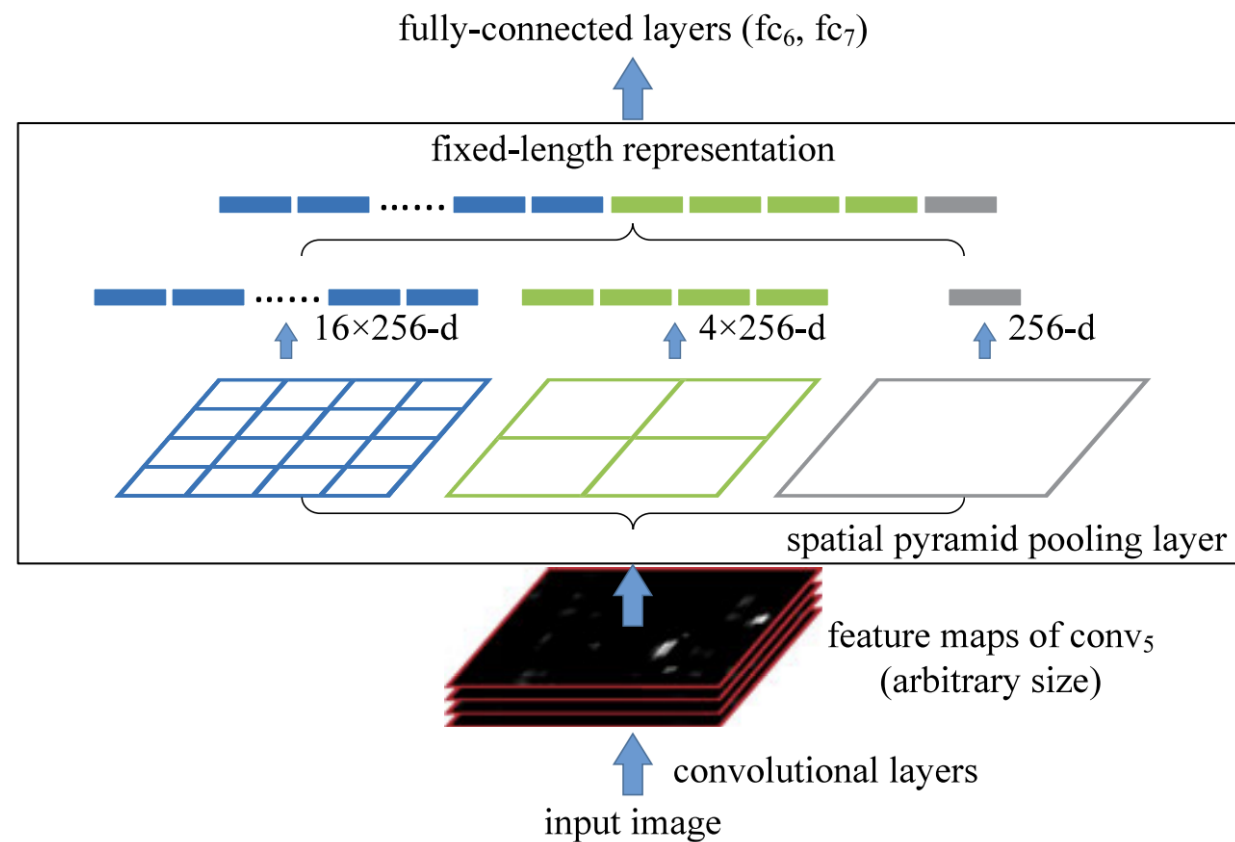
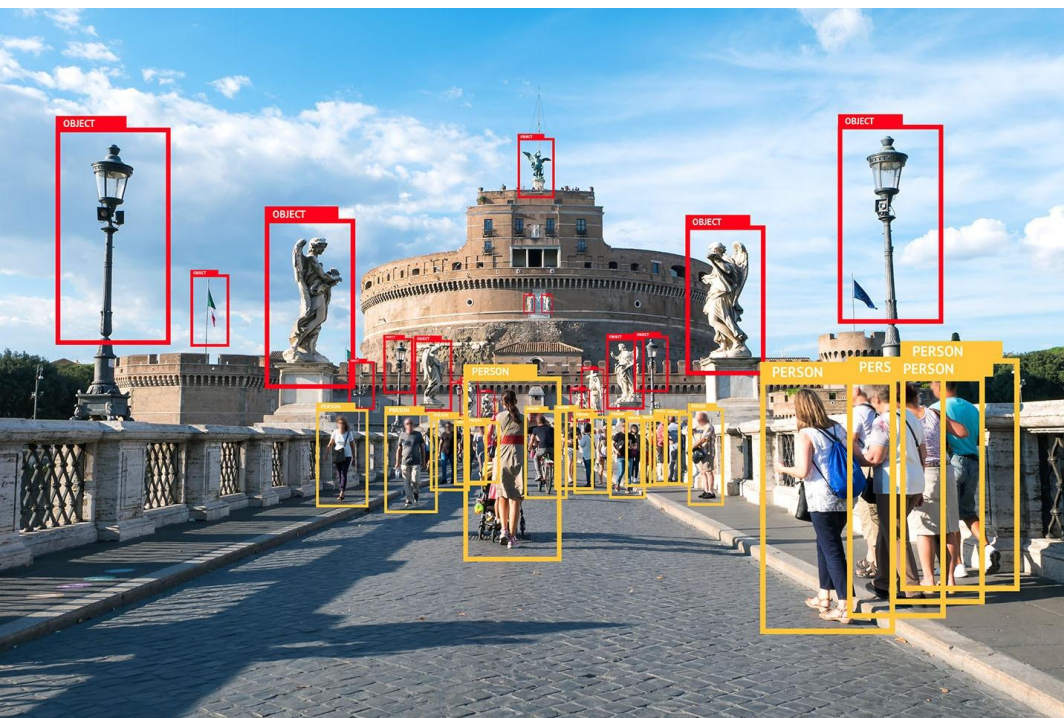
并非总是如此.....当你的模板包含许多重要的高频信息时，它可能会失败。

当你的图像带有噪声时，它可能表现更好。



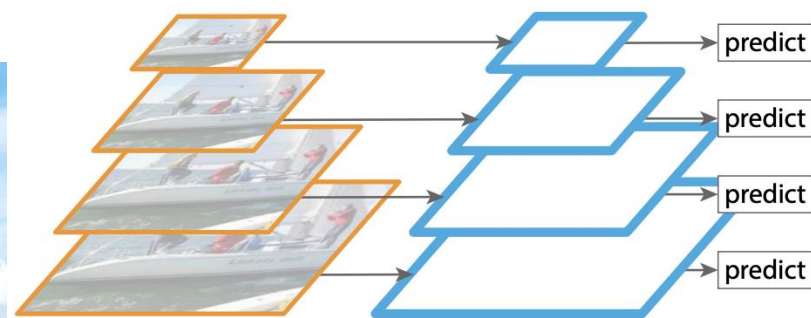
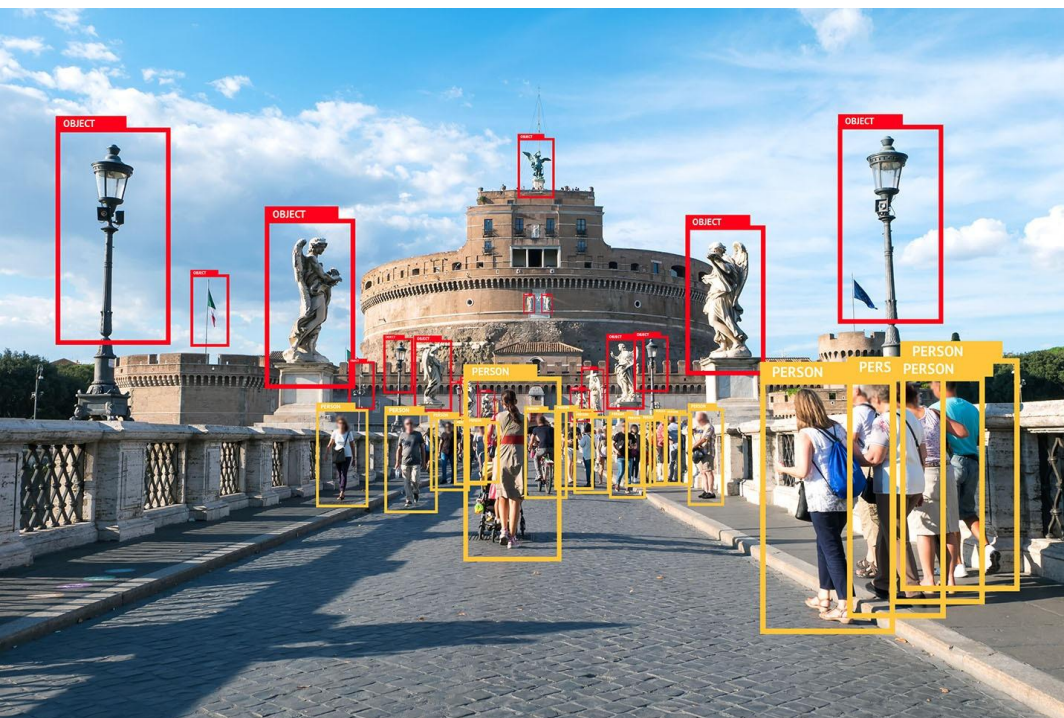
关于图像金字塔的更多内容

目标检测与空间金字塔池化

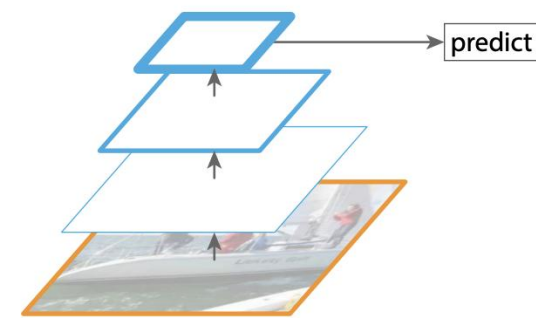


关于图像金字塔的更多内容

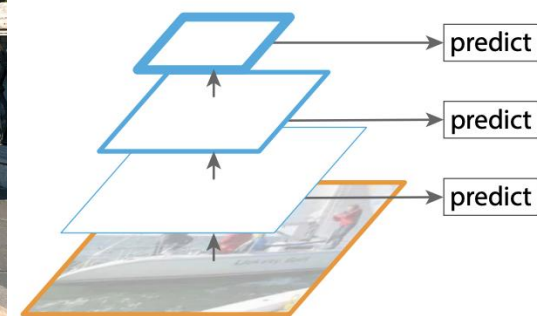
目标检测与特征金字塔网络



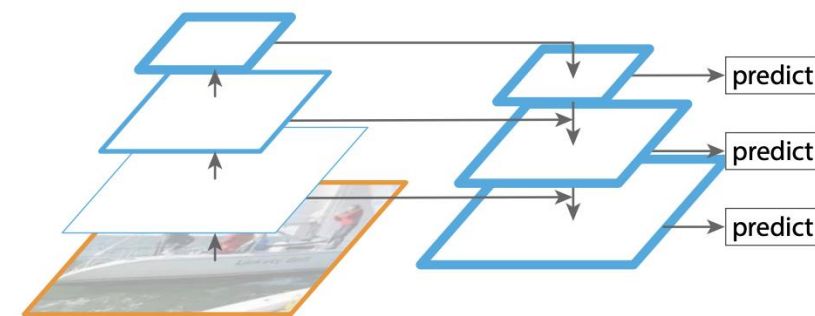
(a) Featurized image pyramid



(b) Single feature map



(c) Pyramidal feature hierarchy

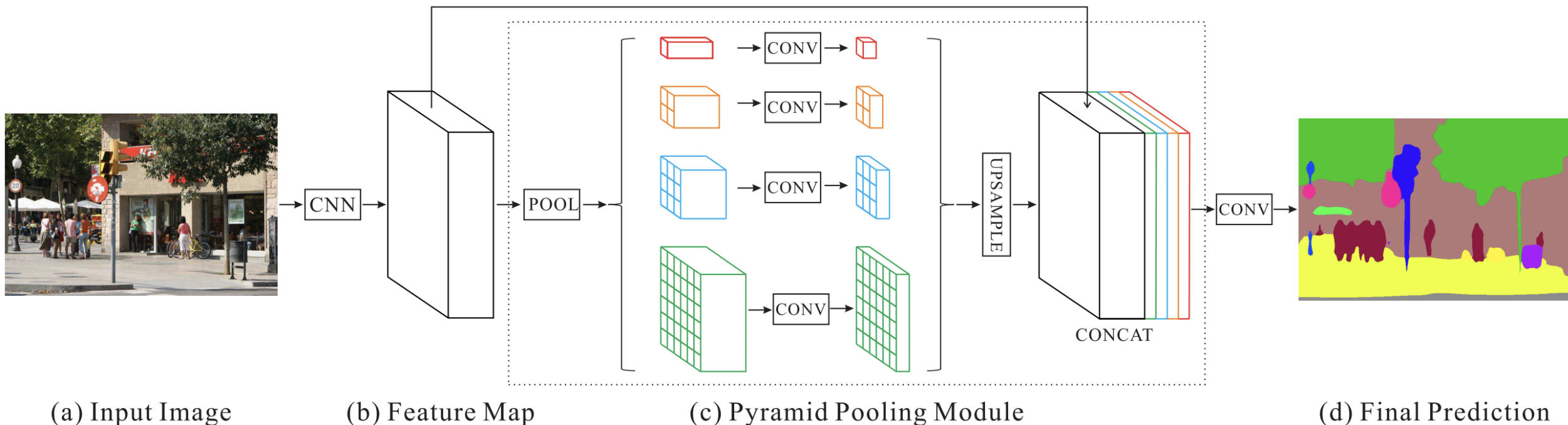


(d) Feature Pyramid Network

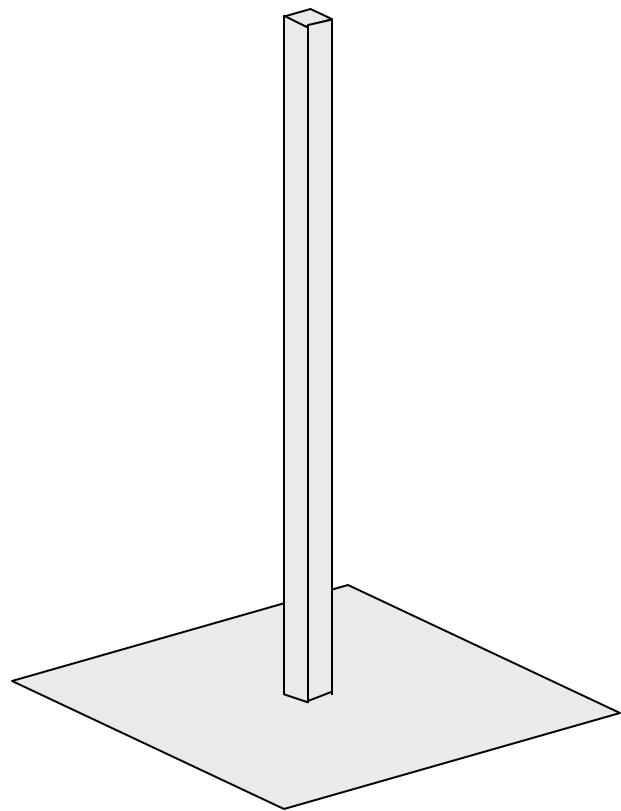


关于图像金字塔的更多内容

图像分割与图像金字塔



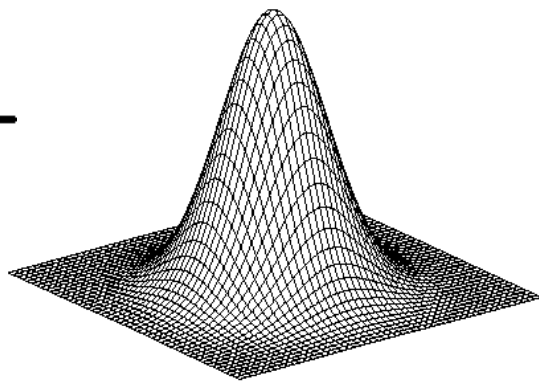
拉普拉斯滤波器



单位冲激

平滑区域

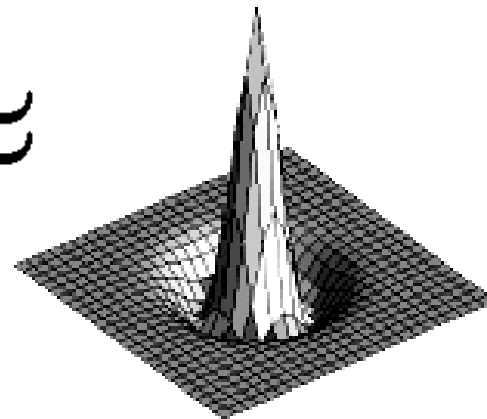
—



高斯

$\approx$

锐化区域，例如边缘



高斯拉普拉斯



拉普拉斯金字塔



512

256

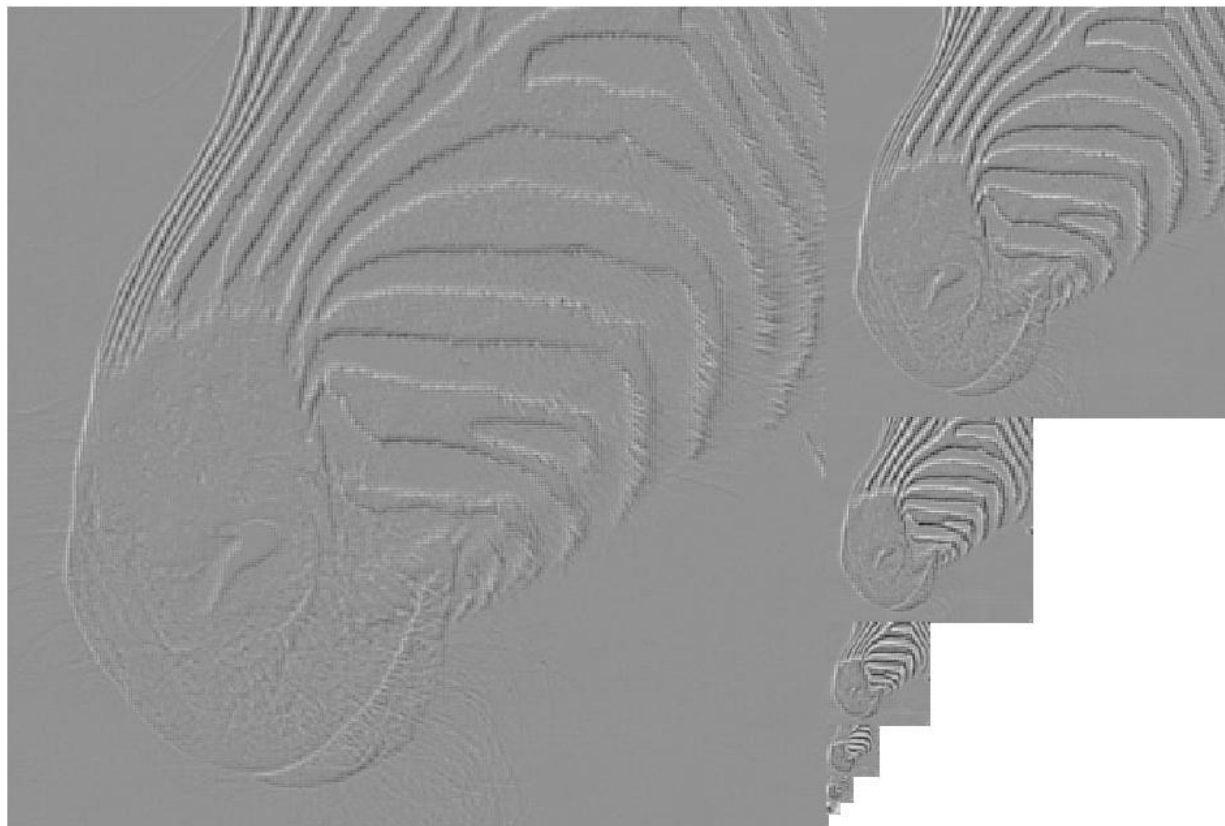
128

64

32

16

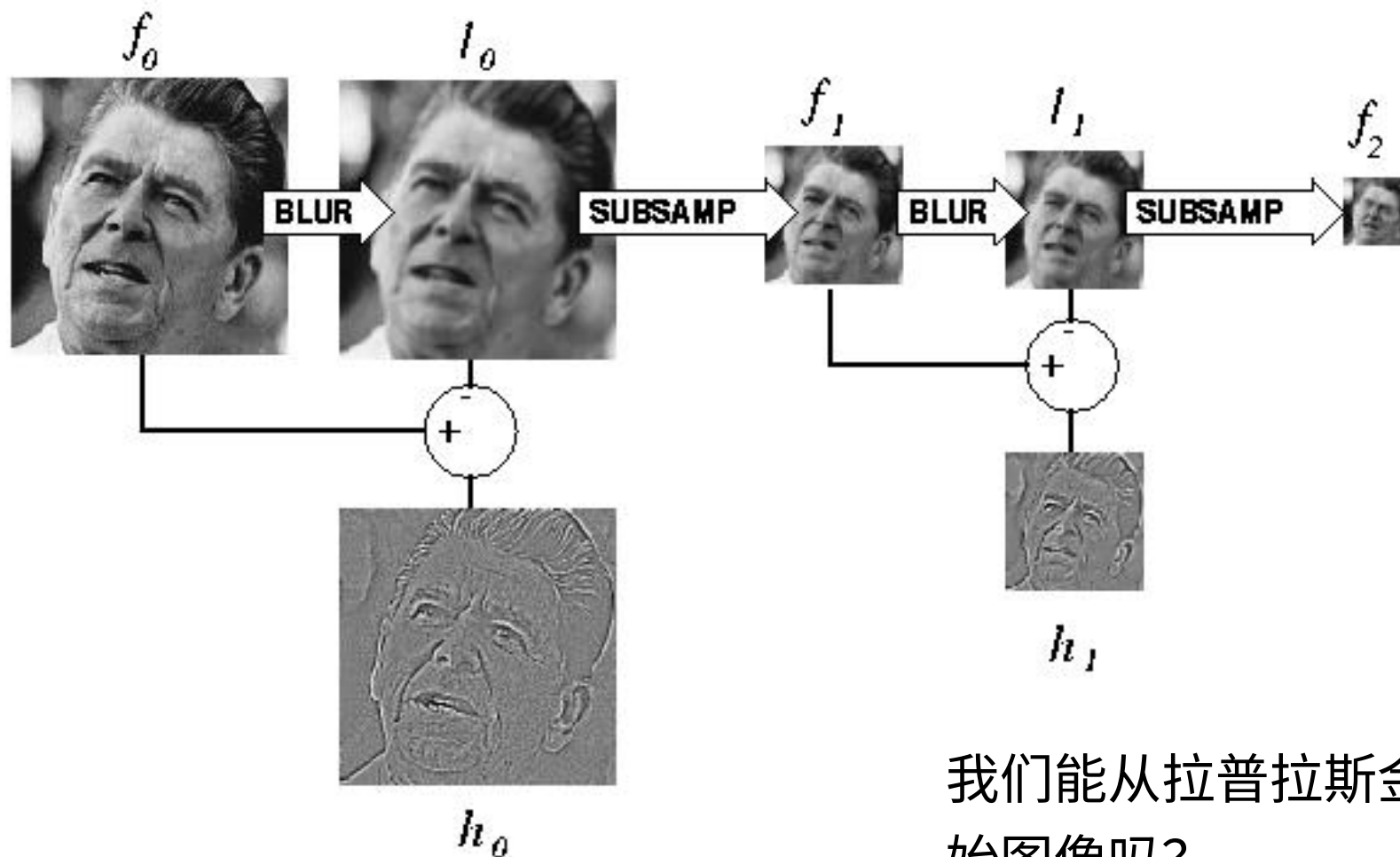
8



来源: Forsyth

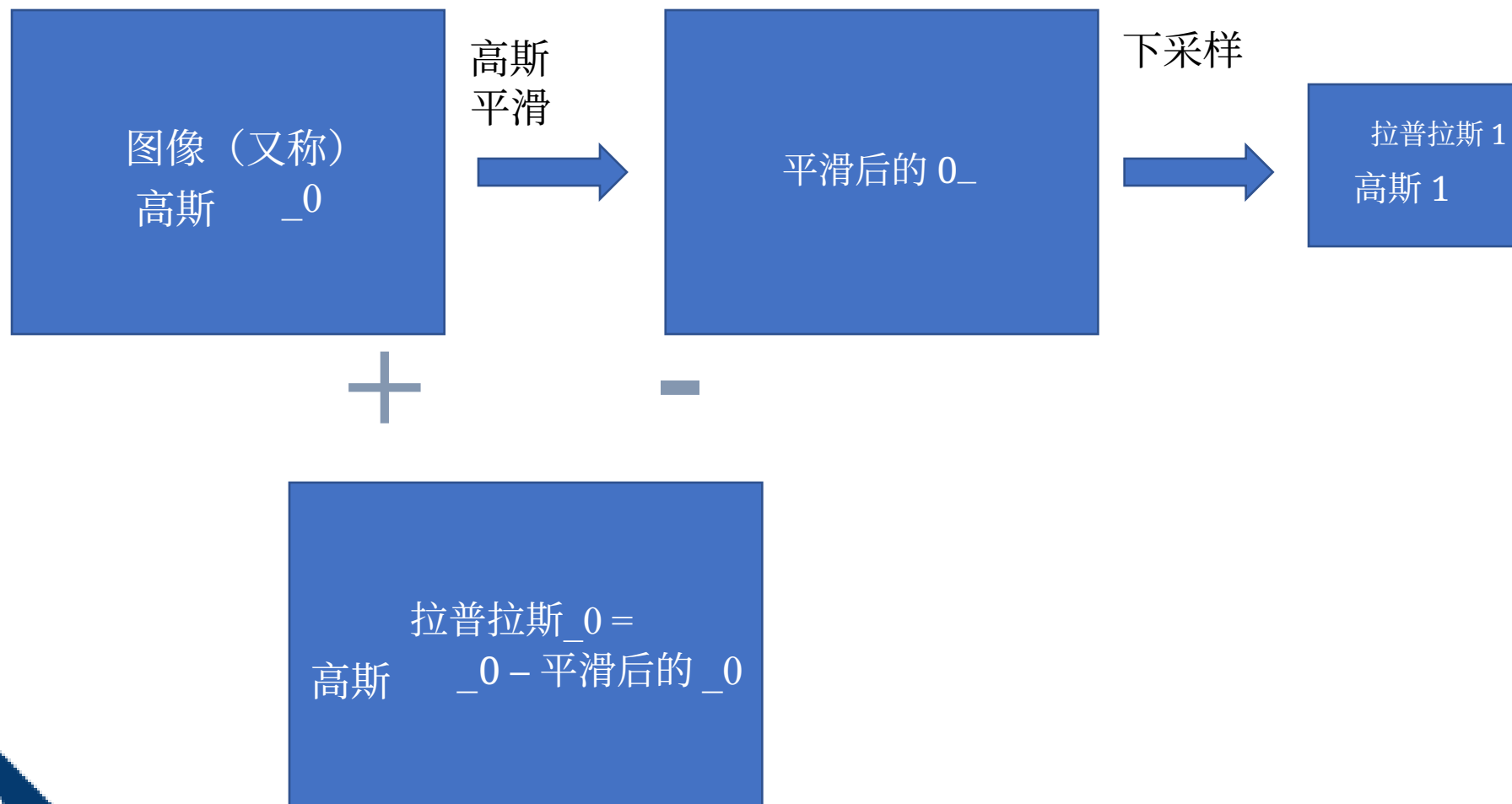


计算高斯/拉普拉斯金字塔

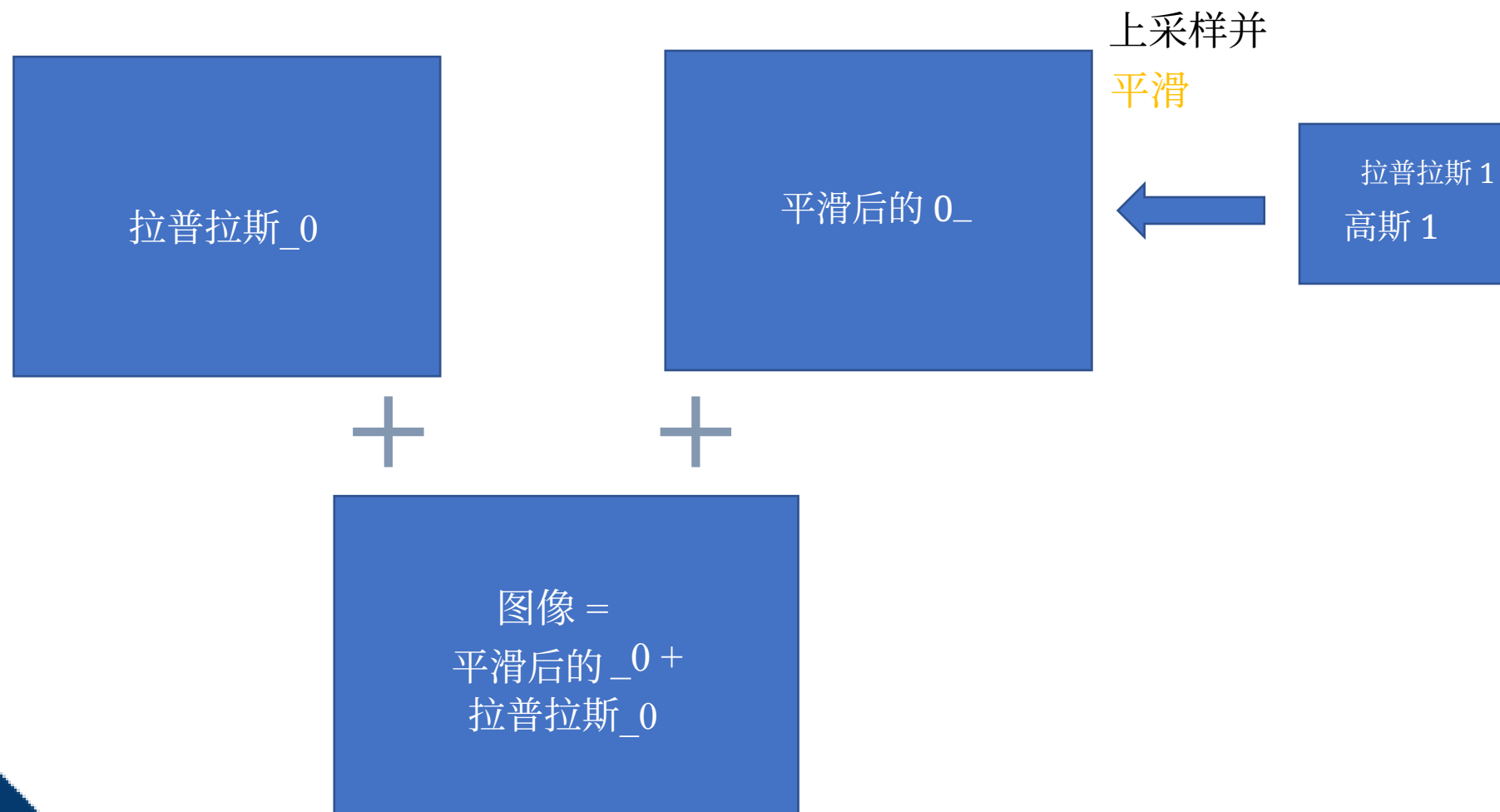


我们能从拉普拉斯金字塔重建原始图像吗？

创建2级拉普拉斯金字塔

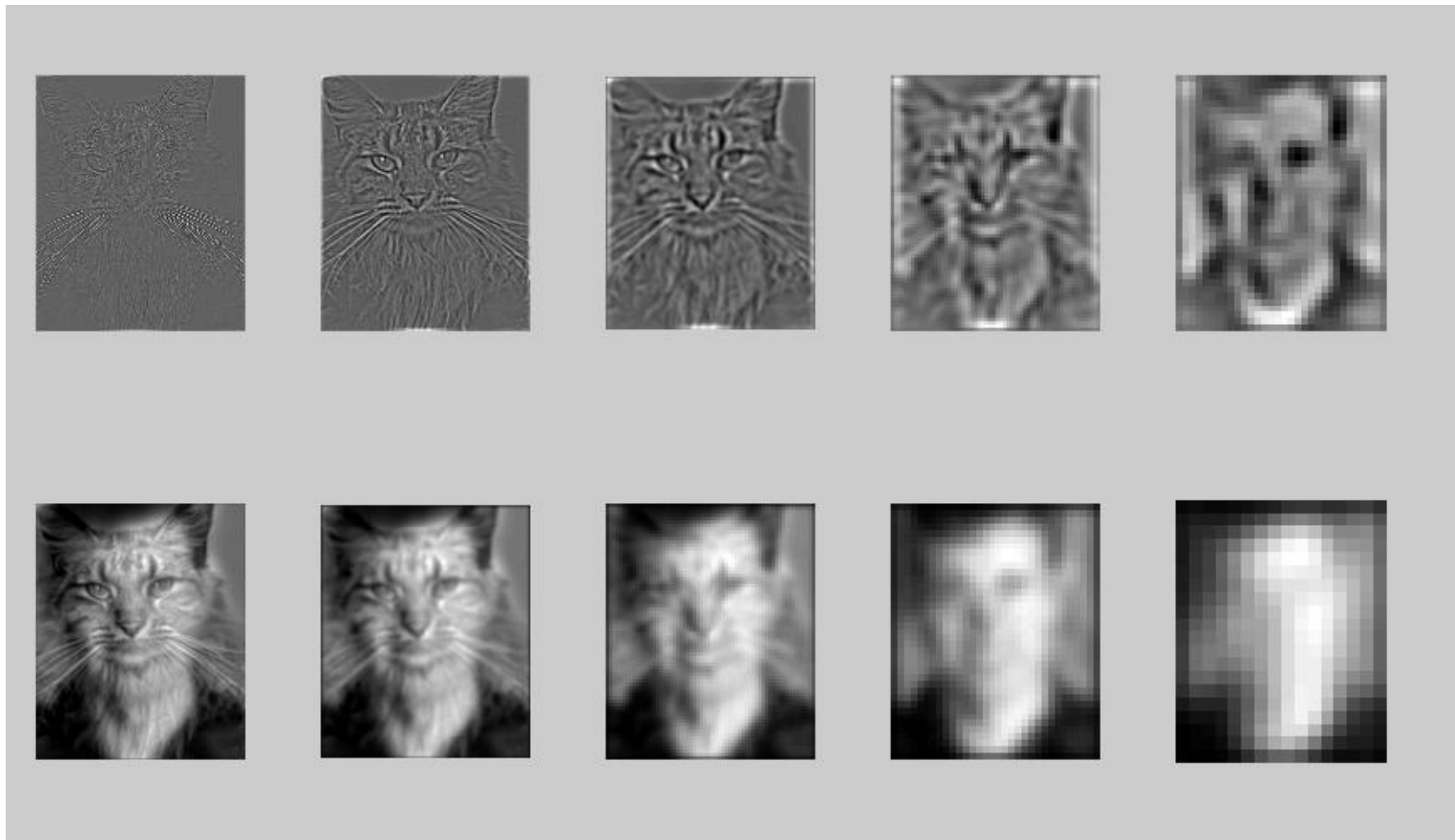


从拉普拉斯金字塔重建图像

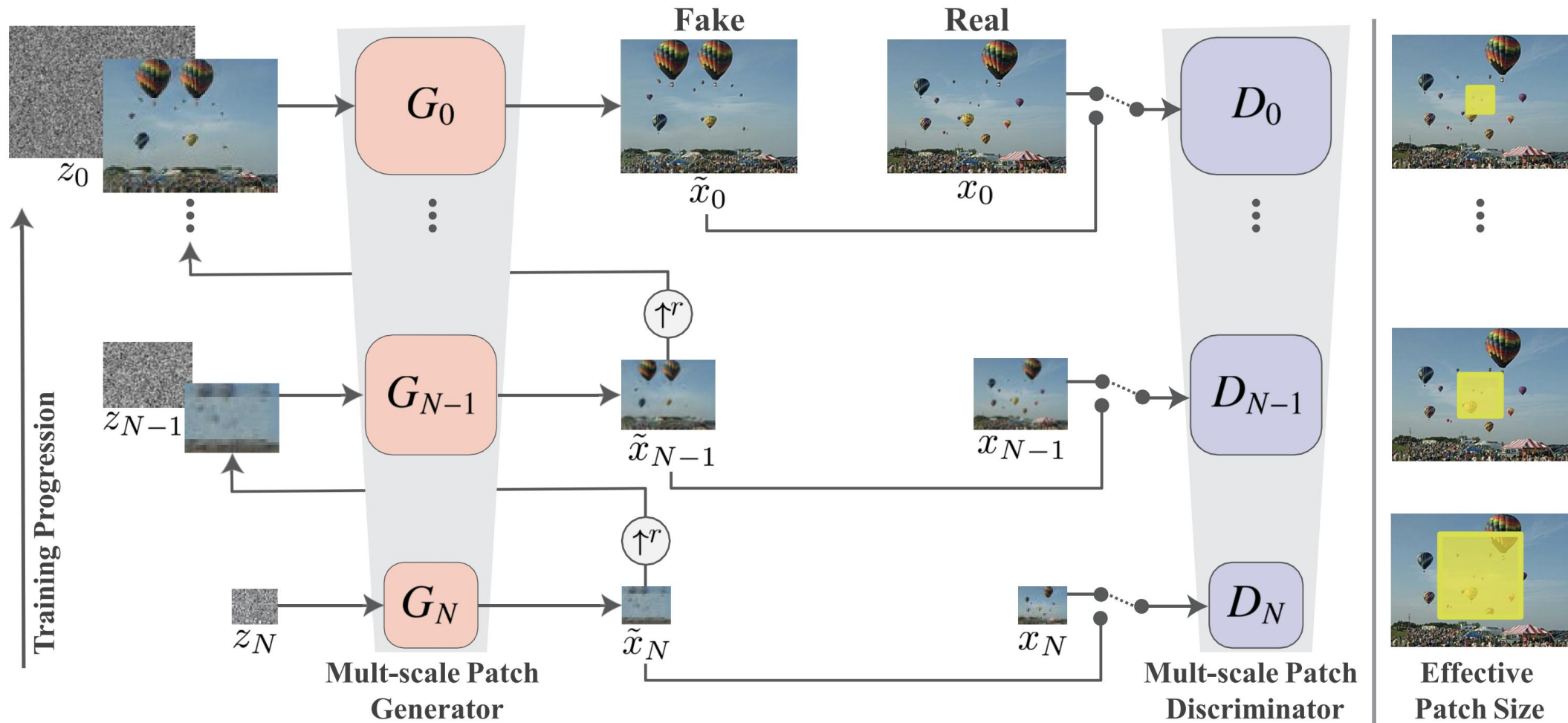


拉普拉斯金字塔中的混合图像

高频 $\rightarrow$ 低频



关于拉普拉斯金字塔的类似思路



要点总结

- 图像滤波器可用于
 - 去噪
 - 图像增强
 - 模板匹配
- 高斯金字塔支持跨不同分辨率进行匹配并提升速度
- 通过进行微小的修改，高斯金字塔就变成了拉普拉斯金字塔，这在图像合成应用中非常有用。



匿名课堂反馈

